

Affordance2Action: Task-Conditioned Scene-level Affordance Grounding for Real-Time Manipulation

Litao Liu^{1,*}, Yifan Han^{1,*}, Pengfei Yi¹, Wenbo Yu¹,
Hanqing Wang², Haoran Du¹, Enze Yuan¹, Zilin Yuan¹,
Ruiling Feng¹, Michael Liu¹, Qi Zhang³, Jingjin Yu^{1,†}

¹Department of Computer Science, Rutgers University-New Brunswick

²The Hong Kong University of Science and Technology (GZ)

³Shanghai AI Laboratory

*Equal contribution. †Corresponding author.

<https://arc-1.github.io/a2a/>

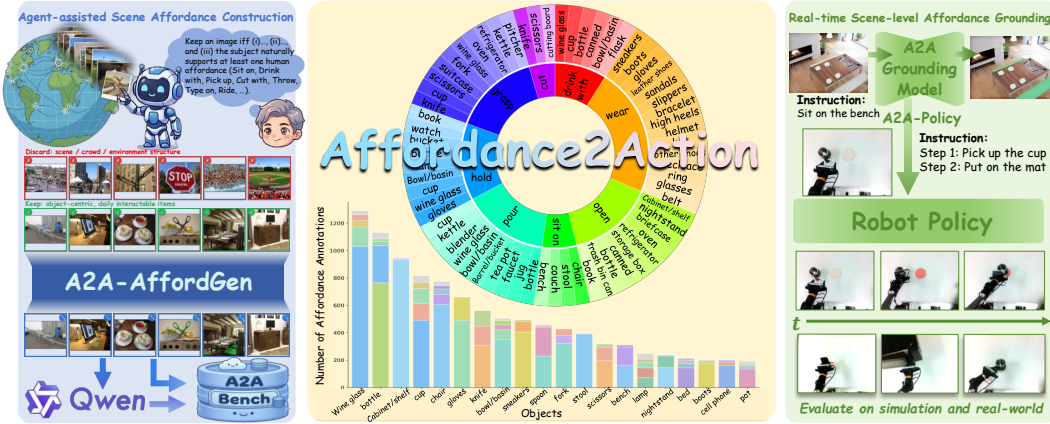


Figure 1. Overview of Affordance2Action (A2A). A2A-AffordGen constructs A2A-Bench with task-conditioned functional-region annotations in natural multi-object scenes. The annotations train A2A-GroundingModel, whose real-time predictions supply spatial priors to A2A-Policy. The annotation teacher is separate from the affordance model used for robot control.

Abstract: Task-conditioned manipulation requires grounding instructions to task-relevant functional parts rather than object categories. This setting is scene-dependent and often one-to-many in cluttered scenes: the same object may afford different interactions across tasks, while a single task may correspond to either one functional region or multiple valid functional regions, depending on the scene layout. Existing affordance datasets and benchmarks remain misaligned with this setting, as they typically focus on grasping or object-level affordances, rely on synthetic scenes, or assume a single instruction-region correspondence. We present **Affordance2Action (A2A)**, a benchmark-centered learning framework for scene-level, task-conditioned part affordance grounding. At its core is **A2A-Bench**, a manipulation-oriented benchmark that covers both single-region and multi-region instruction correspondences in everyday scenes, with the latter highlighting the ambiguity and diversity of affordance grounding in realistic multi-object environments. To construct it at scale, we build **A2A-AffordGen**, an agent-assisted annotation pipeline that combines language-model filtering, interactive part segmentation, instance-level mask-out refinement, task-reasoning instruction generation, and human verification. A2A-Bench’s supervision further supports diverse downstream applications, with real-time affordance grounding and affordance-conditioned manipulation policies as two representative examples. Experiments show that A2A exposes substantial gaps in generic segmentation, VLM-based grounding, and affordance distillation baselines, while improving task-level localization and providing useful spatial priors for downstream manipulation.

Corresponding author: Jingjin Yu (jingjin.yu@cs.rutgers.edu). Project leaders: Litao Liu (litao.liu@rutgers.edu) and Yifan Han (hanyifan2024@ia.ac.cn). This work was completed by Yifan Han, Pengfei Yi, and Wenbo Yu during their internship at Rutgers University-New Brunswick.

1 Introduction

Language-guided manipulation grounds instructions in functional parts that guide action prediction [1]. For example, “open the top drawer of the nightstand” and “place the cup to the nightstand” target different parts of the same object. Grounding depends on task, scene layout, and strategy: instructions may admit multiple valid regions. Vision foundation models [2, 3] segment explicitly described visual concepts, whereas manipulation requires inferring functional regions from task intent. Vision-Language-Action (VLA) policies learn these regions implicitly, reducing interpretability and adaptability. Explicit affordances provide a representation linking language grounding to action prediction.

RAGNet [4] emphasizes grasping; InstructPart [5] links language to parts but lacks object diversity, scene complexity, and scale. UAD [6] reduces annotation costs by distilling foundation-model affordances, but simplified synthetic objects limit robustness in clutter. Most affordance formulations assume one-to-one instruction-region correspondence. Evaluating scene-level understanding instead requires one-to-many annotations covering all plausible task-relevant regions, not only the most salient region.

We propose **Affordance2Action (A2A)** for real-time, task-conditioned part grounding. Its **A2A-Bench** benchmark covers single- and multi-region correspondences in clutter. We filter large-scale natural images [7] for manipulation-relevant affordances and annotate functional regions through interactive segmentation [8] with iterative mask-out refinement. These annotations represent interaction opportunities rather than embodiment-specific execution guarantees, serving as grounding targets and policy priors. Building on this supervision, **A2A-GroundingModel** adapts SAM3 [3] to predict affordance masks from image-instruction pairs without inference-time spatial prompts. Staged instruction adaptation and text-conditioned visual prompts connect explicit part descriptions to task intent. **A2A-Policy** uses these masks as priors for manipulation [9]. Our contributions are:

- **A2A-Bench**: a scene-level benchmark linking task instructions to one or multiple functional regions.
- **A2A-AffordGen**: a pipeline for lower-cost labeling through filtering, interactive segmentation, mask-out refinement, instruction generation, and verification.
- **A2A-GroundingModel** and **A2A-Policy**: real-time part grounding and manipulation models that turn benchmark supervision into spatial action priors.

2 Related Work

Affordance Learning. Affordance learning connects perception and manipulation through 2D masks [6, 10], heatmaps [11, 12], or 3D surface priors from point clouds [13, 14]. HOI-based approaches add temporal cues from pre-contact motion, contact regions, and post-contact dynamics [15, 16]. Foundation models and MLLMs infer functional regions from multimodal inputs [17–20], with Affordance-R1 [21] exploring deep affordance reasoning. Most prior work is object-centric or assumes one-to-one correspondence; A2A-Bench covers single- and multi-region grounding in real-world multi-object scenes.

Affordance for Robot Policies. Language and vision-language models support affordance-aware action reasoning. CoA-VLA [1] decomposes manipulation into object, grasp, spatial, and movement affordances to guide VLA policies. Affordance often remains external to the learned perception-action representation. RAGNet [4] guides grasping with predicted affordance regions. Closest to our work, UAD [6] distills foundation-model affordances into heatmaps for imitation learning. We study both mask highlighting and feature injection as task-conditioned action priors.

Predictive and Generative Policy Learning. Related policy-learning approaches introduce additional structure through prediction, optimization, and data augmentation. FoAM [22] combines multimodal goal conditions with foresight about the visual consequences of actions in multi-task imitation learning. RoboFlow4D [23] predicts multi-frame 3D flows from observations and instructions to guide action generation. DyGRO-VLA [24] studies cross-task VLA optimization through shared representations and grouped residual updates. BiTrajDiff [25] uses forward and backward diffusion of trajectories to augment offline reinforcement-learning data. Our study focuses on functional-part grounding as an explicit intermediate representation and evaluates how masks and region-conditioned features support downstream manipulation.

3 Methodology

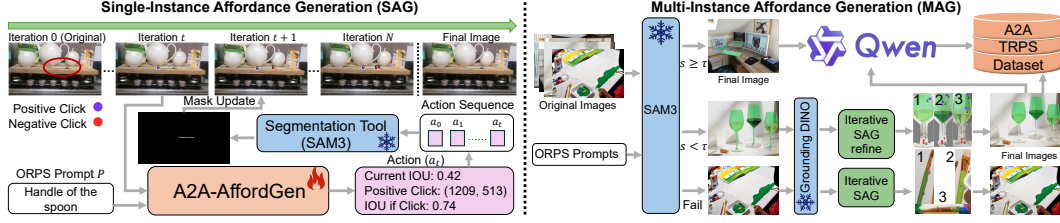


Figure 2. A2A-AffordGen. SAG (left) predicts $\{+, -\}$ clicks from an ORPS prompt and current mask for frozen SAM3. MAG (right) triages SAM3 text-prompted masks, refines low-confidence instances with SAG, and reassembles scene-level multi-instance, multi-part annotations for A2A-TRPS.

3.1 A2A-Bench

One-to-many affordance annotation must identify task-relevant parts across multiple interactable objects in a scene. Although open-vocabulary segmenters such as SAM3 [3] provide strong scene-level grounding, they struggle with small or visually subtle functional parts. We introduce A2A-AffordGen, a dual-regime annotation pipeline for constructing A2A-Bench (Fig. 2), supporting both single-instance and multi-instance affordance generation. Single-Instance Affordance Generation (SAG) refines isolated-object part masks through iterative point prompting; Multi-Instance Affordance Generation (MAG) extends it to cluttered multi-object scenes. Both regimes follow the Oracle Referring Part Segmentation (ORPS) protocol [5], enabling part-level segmentation from general referring prompts. We further use a VLM to author diverse Task Reasoning Part Segmentation (TRPS) instructions [5] from the resulting masks, yielding task-conditioned data for downstream affordance model training (Sec. 3.2).

Click-policy training. We first train the click policy shared by SAG and MAG. Inspired by SegAgent [8], we model point prompting of frozen SAM3 as a Markov decision process. Given crop I , ORPS prompt P , and mask M_t , state s_t contains the mask overlay and prompt. Action $a_t = (p_t, u_t) \in \{+, -\} \times \Omega$ specifies click polarity and pixel; the transition is $M_{t+1} = \Phi(I, P, M_t, a_t)$, where Φ is frozen SAM3 and Ω the pixel grid.

We imitate SimpleClick’s distance-maximizing oracle [26, 27]. For reference mask M^* , false negatives and positives are $E_t^+ = M^* \setminus M_t$ and $E_t^- = M_t \setminus M^*$. The oracle selects the dominant polarity and its chamfer-center:

$$p_t^* = \arg \max_{p \in \{+, -\}} |E_t^p|, \quad u_t^* = \arg \max_{u \in E_t^{p_t^*}} \text{dist}(u, \partial E_t^{p_t^*}), \quad (1)$$

where dist is Euclidean distance to the error boundary. Oracle rollouts start at $M_0 = \emptyset$ and stop at reference IoU κ or step cap $T_{\max}$. Each action $a_t^* = (p_t^*, u_t^*)$ provides next-mask quality and gain targets:

$$v_t^* = \text{IoU}(\Phi(I, P, M_t, a_t^*), M^*), \quad \delta_t^* = v_t^* - \text{IoU}(M_t, M^*). \quad (2)$$

We adapt Qwen3.5-9B [28] with LoRA [29] to emit $(p_t, u_t, \hat{v}_t, \hat{\delta}_t)$ as structured text. Token-level cross-entropy supervises the serialized oracle target $y_t^* = (p_t^*, u_t^*, v_t^*, \delta_t^*)$:

$$\mathcal{L}_{\text{agent}} = \text{CE}(\pi_\theta(\cdot | s_t), y_t^*). \quad (3)$$

The dominant-polarity rule chooses whether omitted or spurious pixels need correction. Selecting a pixel far from the error boundary places the oracle click inside a substantial error region rather than on an uncertain edge. The quality and gain targets are measured after the corresponding SAM3 transition, so they describe the proposed correction rather than the current mask. SAM3 only generates the next state; no pixel-level loss updates the VLM. This division of labor lets the language model choose where and how to correct a mask while the segmenter supplies its pixel geometry. Conditioning each action on the current mask is essential: the appropriate correction depends on the residual error after earlier interactions, rather than on the image and instruction alone.

Single-instance generation (SAG). SAG uses the trained policy to refine a part mask for an object crop and ORPS prompt. Starting from an empty or supplied seed mask, it queries π_θ for a click and predicted next-mask quality and gain. Positive clicks address omitted target regions, and negative clicks suppress incorrectly included regions. At inference, reference masks are unavailable. SAG uses three stopping conditions: (i) Low predicted gain. It stops before clicking if $\hat{\delta}_t < \Delta_{\min}$. (ii) Target IoU reached. If the gain check passes and predicted next-mask IoU $\hat{v}_t > \tau_{\text{iou}}$, SAG applies the click through frozen SAM3 and then stops. (iii) Click budget exhausted. It stops after $T_{\max}$ clicks. Alg. 1 specifies the stopping test and mask update order.

Multi-instance generation (MAG). For scene I and prompt P , SAM3 text prompting supplies candidate masks (m_i, s_i) , while Grounding DINO [30] supplies object boxes. MAG routes instances through three branches: (i) Direct accept ($s_i \geq \tau$). Accept the confident SAM3 mask as-is. (ii) SAG-refine ($s_i < \tau$). Use the low-confidence mask as the seed for SAG within its matched Grounding DINO crop. (iii) SAG-from-scratch (no matched mask). Run SAG from $M_0 = \emptyset$ in each unmatched Grounding DINO box, including scenes where SAM3 finds no mask. For seeded refinement, MAG dilates m_i by kernel $\mathcal{K}$ [31], fills pixels outside $m_i \oplus \mathcal{K}$ with neutral gray, and crops the result and seed mask to its box. Inverse crop mapping returns refined masks $\tilde{M}_i$ to scene coordinates; direct acceptance uses the identity mapping. The scene-level output is

$$\mathcal{M}_P(I) = \bigsqcup_i \mathcal{C}_i^{-1}(\tilde{M}_i), \quad (4)$$

where $\mathcal{C}_i^{-1}$ reverses the corresponding crop transform. Retaining individual masks supports one-to-many supervision rather than reducing a scene to one target. MAG reuses SAG’s single-instance click policy without retraining. Qwen3-VL-32B-Instruct [32] then generates diverse TRPS instructions for task-conditioned grounding. The crop wrapper lets the same single-instance policy operate in clutter without changing its training objective. Dilation preserves a narrow context band around the part, while gray mask-out suppresses neighboring objects that could attract subsequent clicks. Confident predictions bypass refinement; ambiguous and missing instances receive additional interaction.

Algorithm 1 A2A-AffordGen inference: SAG and MAG.

Require: image I , prompt P , policy π_θ , SAM3 Φ , detector G , thresholds $\tau, \tau_{\text{iou}}, \Delta_{\min}$, kernel $\mathcal{K}$, cap $T_{\max}$

```

1: function SAG( $I, P, M$ )
2:   for  $t = 0, \dots, T_{\max} - 1$  do
3:      $(p, u, \hat{v}, \hat{\delta}) \leftarrow \pi_\theta(I, P, M)$ 
4:     if  $\hat{\delta} < \Delta_{\min}$  then
5:       break
6:      $M \leftarrow \Phi(I, P, M, (p, u))$ 
7:     if  $\hat{v} > \tau_{\text{iou}}$  then
8:       break
9:   return  $M$ 
10: function MAG( $I, P$ )
11:    $\{(m_i, s_i)\} \leftarrow \Phi_{\text{text}}(I, P); \mathcal{B} \leftarrow G(I, P)$ 
12:    $\mathcal{M} \leftarrow \emptyset$ 
13:   for each  $(m_i, s_i)$  do
14:     if  $s_i \geq \tau$  then
15:        $\mathcal{M} \leftarrow \mathcal{M} \cup \{m_i\}$ 
16:     else
17:        $b \leftarrow$  box in  $\mathcal{B}$  matched to  $m_i$ 
18:        $I_b \leftarrow$  crop  $b$  after gray mask-out outside  $m_i \oplus \mathcal{K}$ 
19:        $\tilde{M} \leftarrow \text{SAG}(I_b, P, \text{Crop}(m_i, b))$ 
20:        $\mathcal{M} \leftarrow \mathcal{M} \cup \{\mathcal{C}_b^{-1}(\tilde{M})\}$ 
21:   for each unmatched box  $b \in \mathcal{B}$  do
22:      $\tilde{M} \leftarrow \text{SAG}(\text{Crop}(I, b), P, \emptyset)$ 
23:      $\mathcal{M} \leftarrow \mathcal{M} \cup \{\mathcal{C}_b^{-1}(\tilde{M})\}$ 
24:   return  $\mathcal{M}$ 

```

A2A-Bench composition and scale. A2A-Bench is built from large-scale in-the-wild images in three stages (Fig. 1, left). (i) Point-prompting policy training corpus. We curate $\sim 40\text{K}$ single-object

part masks from seven sources, including RAGNet, HANDAL, and InstructPart [4, 33–38], and normalize them into a consistent ORPS format. (ii) Human-verified scene core. From Objects365 [7], we first use VLM-based filtering to discard unsuitable images, such as pure scenes, crowds, or environment-structure images, and retain object-centric daily interactable items. We then manually annotate 5,000 multi-object multi-part scenes spanning diverse tabletop and household categories, including $\sim 18\text{K}$ single-part masks. The masks from (i) and the cropped masks from (ii) are expanded via the SimpleClick distance-maximizing oracle [26] into $\sim 150\text{K}$ click-trajectory samples used to fine-tune A2A-AffordGen. (iii) Automatic scene-level scaling. The trained A2A-AffordGen runs MAG over the unseen Objects365 dataset to auto-generate scene-level annotations. We release an initial batch of 5,000 generated multi-instance scenes after manual quality verification, reducing full manual mask annotation to lightweight human review. The full constructed corpus is used for training; separately annotated test scenes remain held out.

Annotation criteria and one-to-many structure. A valid region corresponds to a functional object part for which contact advances the instruction and is reachable in the depicted scene. This criterion identifies a visual interaction opportunity; it does not guarantee reachability for every robot embodiment or a successful action trajectory. Six trained annotators receive disjoint assignments, and a reviewer checks the annotations for consistency. In the initial 5,000-scene release, 77.2% of scenes contain at least two valid regions. This prevalence motivates retaining separate masks during MAG reassembly and evaluating both instance coverage and count error.

Separation of annotation and deployment. The click policy uses reference masks to construct training trajectories, but receives no reference mask at inference. Likewise, human verification improves the released supervision rather than adding privileged information to the online robot policy. A2A-AffordGen can spend multiple SAM3 interactions on an annotation because it operates during dataset construction. A2A-GroundingModel learns from the resulting supervision and predicts task-conditioned regions without that iterative annotation loop. This teacher–student separation connects annotation quality to deployment efficiency while keeping the two evaluation settings distinct.

The serialized training targets, stopping order, and crop-coordinate handling are detailed in App. A.

3.2 A2A-GroundingModel

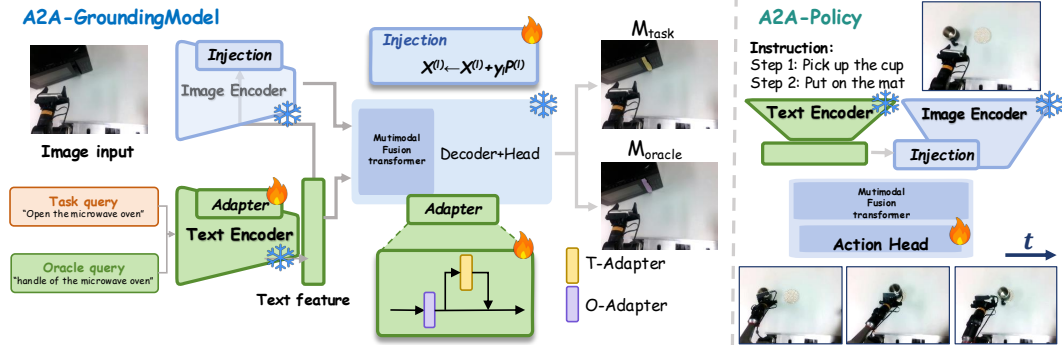


Figure 3. A2A-GroundingModel and A2A-Policy. Lightweight adapters and text-conditioned visual prompts adapt frozen SAM3 for affordance grounding. Explicit mask highlighting or implicit feature injection transfers this prior to manipulation.

A2A-GroundingModel learns real-time functional-region grounding from A2A-Bench supervision (Fig. 3). Given image I and instruction q , it predicts functional-part masks $\hat{M}$ indicating task-relevant interaction regions. Task-conditioned grounding must infer the part implied by manipulation intent: “open the microwave oven” targets its handle rather than the entire appliance. ORPS queries explicitly name the part “handle of the microwave oven”, whereas TRPS queries describe the intended action. Paired ORPS/TRPS supervision connects explicit part localization to task reasoning while retaining the same reference regions.

Staged instruction adaptation. Inspired by SAM3-I [2], we insert lightweight bottleneck adapters into selected language, fusion, decoder, and mask prediction blocks of frozen SAM3. At block ℓ , hidden state $x^{(\ell)}$ follows an oracle route A_o (O-Adapter), or a bridge $A_{o \rightarrow t}$ followed by task adapter

A_t (T-Adapter):

$$\begin{aligned} \mathbf{x}_o^{(\ell)} &= \mathbf{x}^{(\ell)} + A_o^{(\ell)} \left(\text{LN}(\mathbf{x}^{(\ell)}) \right), \\ \mathbf{r}^{(\ell)} &= \mathbf{x}^{(\ell)} + A_{o \rightarrow t}^{(\ell)} \left(\text{LN}(\mathbf{x}^{(\ell)}) \right), \\ \mathbf{x}_t^{(\ell)} &= \mathbf{r}^{(\ell)} + A_t^{(\ell)} \left(\text{LN}(\mathbf{r}^{(\ell)}) \right). \end{aligned} \quad (5)$$

Here LN denotes layer normalization; o and t denote Oracle query and Task query in Fig. 3. Connected routes reduce interference between part recognition and task reasoning; inference uses the task route. The distinction between query types separates two demands on the model. An oracle query supplies the functional-part name, whereas a task query requires selecting that part from the intended action before localizing it. Because the paired queries refer to the same regions, the explicit query provides a localization anchor during adaptation. Deployment uses task instructions without part names, reference masks, or oracle-route predictions.

Text-conditioned visual prompts. Let $\mathbf{T} \in \mathbb{R}^{L \times d_t}$ be language tokens and $\mathcal{I}_q$ their valid, non-padding positions. Masked averaging gives instruction vector $\mathbf{z} \in \mathbb{R}^{d_t}$. Patch projection and a shallow visual network produce image tokens $\mathbf{V} \in \mathbb{R}^{HW \times d_p}$. A learned text bias $\mathbf{b}$ and channel gate $\mathbf{g}$ modulate these tokens using spatial weights α_j :

$$\begin{aligned} \mathbf{z} &= \frac{1}{|\mathcal{I}_q|} \sum_{i \in \mathcal{I}_q} \mathbf{T}_i, \quad \mathbf{b} = W_b \text{LN}(\mathbf{z}), \\ \mathbf{g} &= \sigma(W_g \text{LN}(\mathbf{z})), \quad \alpha_j = \frac{\exp(\mathbf{V}_j^\top \mathbf{b} / \sqrt{d_p})}{\sum_k \exp(\mathbf{V}_k^\top \mathbf{b} / \sqrt{d_p})}, \\ \mathbf{S}_j &= \text{LN}(\mathbf{V}_j \odot (1 + \alpha_j \mathbf{g}) + \mathbf{b}), \quad \mathbf{X}^{(\ell)} \leftarrow \mathbf{X}^{(\ell)} + \gamma_\ell \mathbf{P}^{(\ell)}. \end{aligned} \quad (6)$$

Here σ is sigmoid and $\odot$ element-wise multiplication; the weights α_j normalize over spatial locations. Spatial weighting identifies relevant locations, while the gate modulates feature channels. Layer-specific projections map seed $\mathbf{S}$ to prompts $\mathbf{P}^{(\ell)}$ matching visual features $\mathbf{X}^{(\ell)}$; γ_ℓ is learned. Prompts enter later visual encoder blocks, where semantically structured features support task conditioning. This introduces manipulation intent during visual feature formation, emphasizing small or weakly textured parts that whole-object semantics can obscure. Adapters modify hidden-state processing, while visual prompts add instruction-dependent signals to image tokens. Together they produce task-specific image representations while keeping the backbone frozen.

Training objective and schedule. Hungarian assignment matches predictions to reference masks. The segmentation objective combines classification, box regression, generalized-IoU, and mask losses:

$$\mathcal{L}_{\text{seg}} = \lambda_{\text{cls}} \mathcal{L}_{\text{cls}} + \lambda_{\text{box}} \mathcal{L}_{\text{box}} + \lambda_{\text{giou}} \mathcal{L}_{\text{giou}} + \lambda_{\text{mask}} \mathcal{L}_{\text{mask}}. \quad (7)$$

For paired ORPS/TRPS queries, spatial distributions $\mathbf{P}_o$ and $\mathbf{P}_t$ from the oracle and task routes are aligned by a bidirectional consistency objective:

$$\mathcal{L}_{\text{KL}} = \frac{1}{2} [D_{\text{KL}}(\mathbf{P}_o \| \mathbf{P}_t) + D_{\text{KL}}(\mathbf{P}_t \| \mathbf{P}_o)]. \quad (8)$$

The task-query term supervises $\hat{M}_{\text{task}}$, corresponding to M_{task} in Fig. 3. For reference M and oracle-task disagreement weight $\mathbf{W}_{\text{task}}$ on grid Ω ,

$$\mathcal{L}_{\text{task}} = \frac{1}{|\Omega|} \sum_{u \in \Omega} \mathbf{W}_{\text{task}}(u) \mathcal{L}_{\text{mask}}(\hat{M}_{\text{task}}(u), M(u)). \quad (9)$$

The combined objective is

$$\mathcal{L} = \mathcal{L}_{\text{seg}} + \lambda_{\text{KL}} \mathcal{L}_{\text{KL}} + \lambda_{\text{task}} \mathcal{L}_{\text{task}}. \quad (10)$$

Stage I learns localization from explicit ORPS part descriptions. Stage II transfers this ability to TRPS queries that imply the functional part through task intent. Stage III jointly trains the paired routes with consistency regularization to preserve part localization while aligning task-conditioned predictions. Only adapters and visual prompt modules update; the SAM3 backbone remains frozen. The oracle route supplies training supervision, while inference requires only I and the task instruction.

The step-by-step adapter, visual-prompt, and training formulations are provided in App. B.

3.3 A2A-Policy

We instantiate A2A-GroundingModel (Sec. 3.2) in two manipulation-policy variants. Both variants share an action-chunking diffusion head [9, 39] and use different affordance interfaces. A2A-GroundingModel remains frozen.

Explicit mask highlighting. In the explicit variant, inspired by UAD [6], A2A-GroundingModel grounds task-relevant functional regions on the raw RGB observation. For policy conditioning, we use the highest-confidence grounded region as a colored alpha overlay before feeding the observation to the action head. The remaining policy matches the diffusion baseline, isolating the highlighting effect.

Implicit feature injection. In the implicit variant (Fig. 3, right), the frozen A2A-GroundingModel acts as the visual-text encoder. We process each frame with text-conditioned visual prompt injection (Sec. 3.2). We select the highest-confidence predicted region and threshold its mask before pooling the multi-scale FPN features [40], following Afford-VLA [41]. Region-conditioned tokens are projected to the policy feature width. A shared multimodal transformer fuses them with adapted instruction and state tokens; only its $\langle \text{ACT} \rangle$ token conditions the diffusion head. This replaces unfiltered coarse feature pooling while keeping the training setup unchanged. Affordance guides feature formation and spatial aggregation without overlays.

The two variants test where to introduce the same grounding prior: explicit conditioning modifies the visual observation, whereas implicit conditioning restricts the features passed to the policy. Freezing the grounding model keeps policy learning from changing its masks to fit individual demonstrations. Selecting one region is a deliberate interface simplification for these manipulation tasks; it does not evaluate a planner that reasons over or acts on all regions produced by the scene-level model.

4 Experiments

We evaluate the framework at three levels: annotation quality from A2A-AffordGen, task-conditioned prediction from A2A-GroundingModel, and downstream manipulation with A2A-Policy. These evaluations use different inputs and outputs. ORPS supplies an explicit part description to assess mask generation, TRPS asks a model to infer the relevant part from a task instruction, and policy evaluation measures whether the robot completes that task. We therefore report each protocol separately rather than treating a high grounding score as a substitute for successful execution.

4.1 Benchmarks, Metrics, and Implementation

Grounding benchmarks. We evaluate A2A-AffordGen on separately annotated, held-out ORPS sets: 200 single-instance SAG scenes and 200 multi-instance MAG scenes. The same examples are used across seeds 0, 1, and 2, and the test images are excluded from the constructed training corpus. We also evaluate A2A-GroundingModel on these ORPS sets to compare the annotation teacher with the real-time student under the same query type. Separately, online task-conditioned grounding is evaluated on 200 fixed TRPS mask-query pairs, with mask accuracy, latency (ms/image), and throughput (FPS). Zero-shot transfer uses RefCOCO, RefCOCO+, and RefCOCOg [42–44], reporting referring-segmentation cIoU without benchmark-specific fine-tuning.

Mask-overlap and threshold metrics. For an evaluated prediction–reference pair $(\hat{M}_n, M_n^*)$, let $I_n = |\hat{M}_n \cap M_n^*|$, $U_n = |\hat{M}_n \cup M_n^*|$, and $J_n = I_n/U_n$. We report

$$\text{gIoU} = \frac{1}{N} \sum_{n=1}^N J_n, \quad \text{cIoU} = \frac{\sum_{n=1}^N I_n}{\sum_{n=1}^N U_n}. \quad (11)$$

Here gIoU is the mean of the evaluated mask-pair IoUs, not generalized box IoU. It weights mask pairs equally, whereas cIoU pools intersection and union pixels and therefore gives larger unions more weight. Define $P(t) = N^{-1} \sum_n \mathbf{1}[J_n \geq t]$. P@50 is $P(0.50)$ and P@50–95 is the mean of $P(t)$ over the ten thresholds $t = 0.50, 0.55, \dots, 0.95$. Higher thresholds require more precise masks. Grounding accuracy values are computed per run and averaged over the three seeds; all overlap and threshold-based scores are presented as percentages.

Multi-instance set metrics. Pixel overlap alone does not reveal whether a method has recovered every valid instance. Let K_p and K_g be the numbers of predicted and reference masks in a scene,

and set $K = \max(K_p, K_g)$. We pad the smaller set with empty masks and give padded pairs zero IoU. The set-overlap score is

$$\text{sIoU} = \max_{\sigma \in \mathfrak{S}_K} \frac{1}{K} \sum_{k=1}^K \text{IoU}(\hat{M}^{(k)}, M^{*(\sigma(k))}), \quad (12)$$

where $\mathfrak{S}_K$ contains the permutations and Hungarian matching finds the optimum. One prediction can match at most one reference; padding penalizes missed and extra regions. If T_n real matched pairs have IoU at least 0.5 in scene n , set recall is $R_n = T_n/K_{g,n}$ (Set-R), and set precision is $P_n = T_n/K_{p,n}$ (Set-P); padded pairs never count as successful matches. Count MAE is $N^{-1} \sum_n |K_{p,n} - K_{g,n}|$. Set-R measures instance coverage, Set-P measures the fraction of useful proposals, and Count MAE measures count error irrespective of boundary shape. These measures complement rather than replace pixel overlap. The reported set scores are percentages, while Count MAE is an instance count with lower values better.

Policy benchmarks and success metric. Policy evaluation uses task completion rather than segmentation overlap. Each of ten LIBERO-Object tasks [45] has 100 expert demonstrations and 50 evaluation episodes; we report mean task success rate. Four real-world tasks on an AgileX Piper six-DoF arm each use 100 teleoperated demonstrations and ten evaluation trials with wrist/head RGB observations. The real-robot aggregate is the total number of successful trials out of 40. These counts are kept separate from the three-seed grounding averages.

Grounding implementation. All training and deployment use NVIDIA RTX PRO 6000 Blackwell. A2A-AffordGen fine-tunes Qwen3.5-9B with LoRA and permits at most eight clicks per instance. Frozen SAM3 converts the predicted clicks into masks. A2A-GroundingModel trains adapters and visual prompts within frozen SAM3, with the task route used at deployment. A2A-AffordGen generates supervision; the real-time A2A-GroundingModel supplies the affordance signal to both policy variants in simulation and on the robot.

Policy training and execution. Both A2A-Policy variants use the same frozen A2A-GroundingModel checkpoint across simulation and reality. The three DINOv2 [46] variants use 448×448 RGB inputs, with region overlays for explicit conditioning. Policy overlays use threshold 0.7, separately from the threshold used to evaluate UAD grounding. Implicit conditioning pools thresholded top-1 A2A-GroundingModel features at 336×336 in simulation and 448×448 on the robot. All four policies share a four-layer multimodal Transformer fusion encoder (width 1024, eight attention heads, feed-forward width 2048) and a depth-six, width-768 diffusion head [9, 39]. They use two-frame history, seven-dimensional actions, normalized 16-step action chunks, and 100 training diffusion steps. For LIBERO, all variants train for 30 epochs with AdamW (learning rate 10^{-4}) and per-GPU batch size 16. Policies replan every eight steps using eight DDIM steps. On the Piper arm, each baseline learns one policy across the four tasks, using head/wrist RGB and frozen MiniLM-L6-v2 language features.

App. E gives the full policy architecture, optimization schedule, and robot deployment interface.

4.2 A2A-AffordGen: Annotation Quality and Efficiency

Single-instance grounding. Tab. 1(a) compares text/box prompting, iterative clicking, and mask decoding under the ORPS protocol. A2A-AffordGen achieves 81.91 gIoU, 80.55 cIoU, 90.50 P@50, and 75.00 P@50–95, leading all four columns. Its gIoU exceeds finetuned LISA-7B by 13.66 points. The improvement also persists at stricter overlap thresholds: P@50–95 exceeds finetuned LISA by 18.97 points. Thus, the difference is not restricted to getting a coarse region above the 0.5 threshold.

A2A-GroundingModel achieves 72.78 gIoU under the same ORPS protocol, between finetuned LISA and the annotation teacher. The teacher–student gIoU gap is 9.13 points. Their roles differ: A2A-AffordGen prioritizes annotation quality and takes approximately 15.7 seconds per query, whereas A2A-GroundingModel supports online grounding. Evaluating the student on ORPS isolates this quality comparison; its intended task-instruction interface is evaluated separately in Sec. 4.3.

Multi-instance grounding. Tab. 1(b) reports all baselines on the expanded 200-scene ORPS evaluation. Its overlap metrics measure mask agreement, while the set metrics additionally test coverage, proposal precision, and instance counts. A2A-AffordGen reaches 71.25 gIoU and 73.82 cIoU, together with 67.85 sIoU, 73.88% set recall, and 71.56% set precision. Its count MAE is 1.23, the lowest among the listed methods.

Table 1. ORPS grounding on A2A-Bench, with 200 held-out scenes per protocol. Results averaged over three seeds; percentages except Count MAE.

(a) Single-instance grounding								
Method	gIoU	cIoU	P@50	P@50–95				
SAM3+text [3]	44.58	46.07	48.00	38.70				
GroundingDINO+SAM3 [30]	41.87	30.27	41.17	29.65				
Qwen3.5-9B+SAM3 [28]	57.04	71.97	61.83	46.83				
Qwen3-VL-32B+SAM3 [32]	59.25	63.33	66.83	50.73				
SegAgent [8, 47, 48]	21.62	19.51	17.83	10.10				
LISA-7B [49]	25.83	31.27	24.67	13.50				
LISA-7B (finetuned) [49]	68.25	76.44	74.67	56.03				
A2A-GroundingModel (student)	72.78	78.66	76.67	57.92				
A2A-AffordGen (teacher)	81.91	80.55	90.50	75.00				

(b) Multi-scene grounding								
Method	gIoU	cIoU	P@50	P@50–95	sIoU	Set-R	Set-P	Count MAE ↓
SAM3+text [3]	60.84	65.98	62.50	57.60	53.56	53.33	65.56	1.73
GroundingDINO+SAM3 [30]	40.25	29.14	32.83	23.88	30.33	21.67	33.15	1.98
Qwen3.5-9B+SAM3 [28]	24.48	32.14	23.17	15.10	21.75	24.32	47.13	2.60
Qwen3-VL-32B+SAM3 [32]	38.99	59.78	41.17	24.75	32.45	33.17	55.10	2.10
SegAgent [8, 47, 48]	29.23	23.84	16.83	11.40	21.51	12.15	24.47	2.03
LISA-7B [49]	33.32	31.60	21.83	17.55	22.82	21.97	21.39	2.46
LISA-7B (finetuned) [49]	53.18	61.40	57.50	43.92	40.74	43.23	34.96	2.29
A2A-GroundingModel (student)	63.44	68.30	66.50	54.57	56.77	56.32	62.67	1.60
A2A-AffordGen (teacher)	71.25	73.82	79.50	67.15	67.85	73.88	71.56	1.23

Relative to A2A-GroundingModel, the teacher gains 11.08 sIoU points and 17.56 set-recall points, while reducing count MAE from 1.60 to 1.23. Relative to native SAM3 text prompting, its sIoU improves by 14.29 points and set recall by 20.55 points. The student also improves overlap and recall over native SAM3, but its set precision of 62.67% is below SAM3’s 65.56%. This distinction matters: improved coverage can coexist with additional spurious proposals. The full pipeline improves both recall and precision over SAM3, supporting the use of instance refinement and mask-out for annotation. Finetuned LISA improves markedly over its untuned counterpart, but remains behind the teacher on the multi-instance overlap and set metrics.

Component ablations. Tab. 2 removes one component at a time from the full multi-instance pipeline. Replacing SAM3 text grounding with GroundingDINO boxes causes the largest drop, 24.5 gIoU points, indicating that the initial mask proposals are an important part of the pipeline. Removing iterative clicking costs 10.4 points. Removing direct acceptance costs 4.7 points, while removing the SAG-refine seed mask costs 4.3 points. These results support preserving reliable initial predictions and using uncertain masks as refinement seeds, rather than treating every candidate as a fresh segmentation problem.

Table 2. Component ablations of A2A-AffordGen on multi-scene ORPS grounding. Each row changes one component of the full pipeline (71.25 gIoU); the reported decrease is in percentage points. Negative values denote lower accuracy.

Change to the full pipeline	Δ gIoU
Replace SAM3 text grounding with GroundingDINO boxes	−24.5
Remove iterative clicking	−10.4
Remove direct acceptance	−4.7
Remove the SAG-refine seed mask	−4.3
Remove dilated gray mask-out	−2.5
Reduce the click cap from eight to two	−3.2

Removing dilated gray mask-out costs 2.5 points. This is consistent with the role of local context control: nearby objects can distract a part-level click policy, while dilation preserves a narrow context band around the current region. Reducing the maximum click count from eight to two costs 3.2 points, indicating that some difficult regions require repeated correction. The ablations measure changes in the complete pipeline; they do not imply that the individual drops add linearly.

Human verification and correction. A separate audit of 500 SAG and 500 MAG masks measures annotation usability. Reviewers accept 76.6% and 65.4% directly, with 4.5 and 10.4 seconds of review per mask, respectively. App. D reports all decision rates, MAG branch proportions, and the verification interface. It distinguishes automatic refinement from human correction, and audit recall from the IoU-thresholded set metric.

Training curves and their interpretation are provided in App. A.4.

4.3 A2A-GroundingModel and Zero-shot Transfer

Task-conditioned grounding and latency. A2A-GroundingModel directly maps an RGB image and a TRPS instruction to functional-region masks. Tab. 3 compares it with A2A-Bench/TRPS-trained UAD [6], SAM3-I [2], and SAM3-LoRA [50]. Unlike ORPS, the query names an intended action rather than explicitly naming the target part. This evaluation therefore tests whether the model identifies the relevant functional region from manipulation intent.

UAD adaptation. UAD retains its heatmap architecture and is trained using merged binary masks with BCE+Dice supervision and no target blur. For each query, the union of positive instance masks is resized to the output grid. Dice has weight 1.0 and smoothing $\epsilon = 1.0$; 448×448 inputs yield 32×32 heatmaps. Training uses Adam for 50 epochs, batch size 128, initial learning rate 2×10^{-3} , and a decay factor of 0.75 every ten epochs. Grounding evaluation binarizes the heatmap at 0.1. This threshold belongs to segmentation evaluation and differs from the 0.7 overlay threshold used by the downstream policies. App. C gives the loss definitions, training settings, and original UAD diagnostic experiments; those earlier diagnostics are reported separately from the current benchmark.

Table 3. Task-conditioned TRPS grounding on 200 fixed mask–query pairs, averaged over three seeds. Accuracy scores are percentages. Latency and throughput are measured at 640×480 input resolution; lower latency and higher FPS are better.

Method	gIoU	cIoU	P@50	P@50–95	ms ↓	FPS ↑
UAD [6]	28.45	36.39	28.17	10.03	62	16.1
SAM3-I [2]	50.99	47.03	51.00	35.10	101	9.9
SAM3-LoRA [50]	52.98	46.26	53.67	36.97	105	9.5
A2A-GroundingModel	55.41	49.52	56.83	39.55	103	9.7

A2A-GroundingModel leads the four reported accuracy metrics, reaching 55.41 gIoU and 49.52 cIoU at 103 ms per image (9.7 FPS). Against SAM3-LoRA, it improves gIoU by 2.43 points and cIoU by 3.26 points with a similar reported latency. UAD is faster at 62 ms (16.1 FPS), but its gIoU is 28.45. These results expose the accuracy–latency tradeoff rather than identifying one method as best on every axis. The reported inference rate measures grounding, not complete closed-loop robot execution.

Fig. 4 illustrates the distinction between diffuse heatmap responses and task-aligned part masks. Small handles, seats, and object contact regions require localization within an object, rather than merely identifying its category. The examples visualize typical spatial differences, while aggregate performance is measured on the held-out set.

Live real-world grounding. Fig. 5 shows predictions as scene content, the instruction, and the camera view change. The first sequence adds mugs while the instruction remains fixed, testing whether the model identifies multiple relevant handles. The second changes the target from bottle caps to a knife handle through an instruction change. The third changes the viewpoint around chairs while targeting their seats. These demonstrations illustrate live responses to changing observations; they are qualitative sequences, not a separate success-rate benchmark.

Zero-shot referring segmentation. We evaluate unchanged A2A-AffordGen and A2A-GroundingModel checkpoints on RefCOCO, RefCOCO+, and RefCOCOg without fine-tuning on

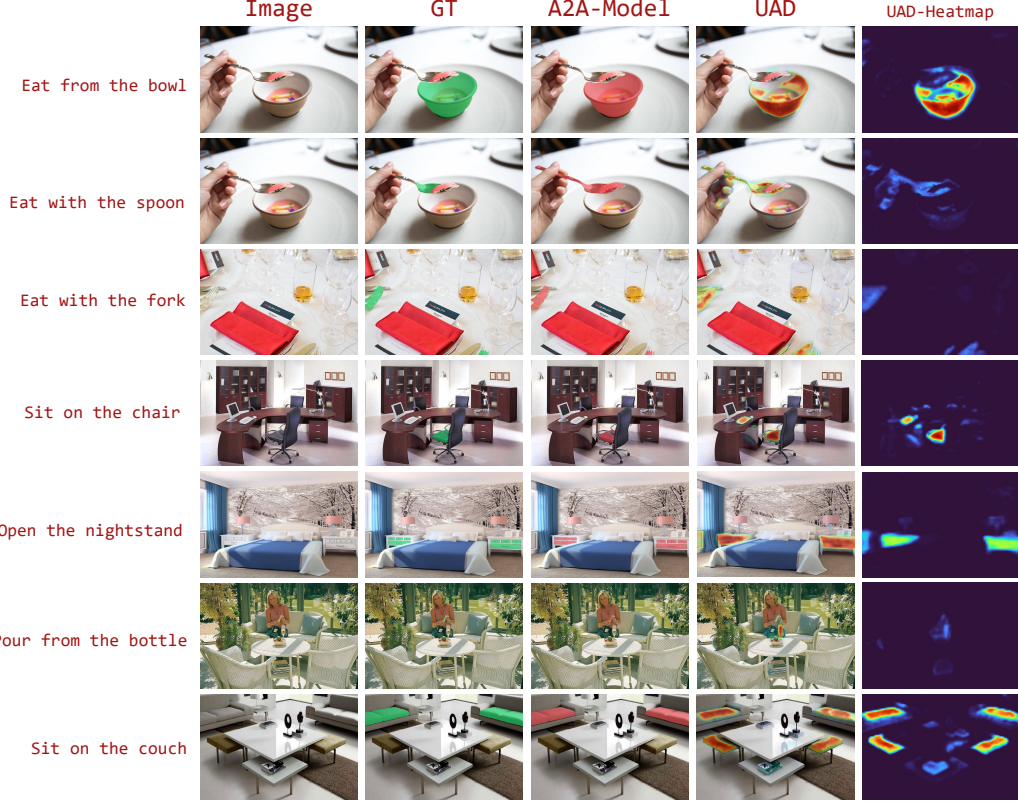


Figure 4. Task-conditioned masks on A2A-Bench. The seven examples show the input image and task, ground-truth mask, A2A-GroundingModel prediction (labeled A2A-Model), binarized UAD prediction, and UAD heatmap. Tasks include eating, sitting, opening, and pouring. The heatmaps expose diffuse responses on unrelated regions, complementing the quantitative TRPS evaluation.

these datasets. Tab. 4 separates the zero-shot methods from models trained with benchmark-specific supervision. The fine-tuned group contextualizes performance but does not share the same training regime.

A2A-AffordGen reaches 75.86, 64.78, and 72.55 cIoU on the three benchmarks, exceeding text-prompted SAM3 by 36.46, 35.34, and 39.79 points. A2A-GroundingModel reaches 47.34, 37.28, and 48.66, improving over the same baseline by 7.94, 7.84, and 15.90 points. The student retains positive transfer relative to its underlying text-prompted baseline, although a substantial teacher-student gap remains.

Among the listed fine-tuned models, A2A-AffordGen exceeds LAVT, LISA, PixelLM, and PerceptionGPT on RefCOCO and RefCOCOg. On RefCOCO+, it is 0.32 points below LISA and below several other fine-tuned baselines. Thus, the transfer is useful across all three datasets without implying uniform superiority over benchmark-specific training. Moving from functional-part supervision to whole-object referring expressions tests a different target granularity, and the results suggest that the learned click procedure transfers beyond the part phrases used to construct A2A-Bench.

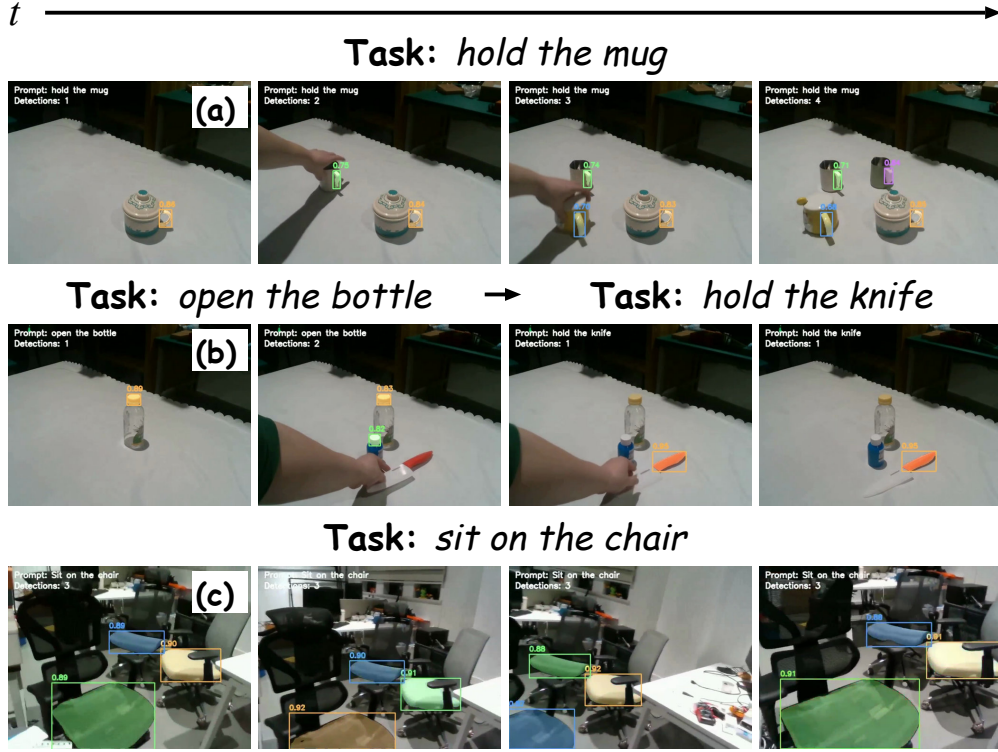


Figure 5. Live A2A-GroundingModel demonstrations. (a) Additional mugs introduce new target handles. (b) The instruction changes from opening bottles to holding a knife. (c) Camera motion changes the chair views. Frames progress from left to right within each sequence.

Table 4. Referring segmentation (cIoU, %). Fine-tuned methods use benchmark-specific training and provide context; the zero-shot group uses unchanged checkpoints without RefCOCO-family fine-tuning. The two groups represent different supervision regimes.

Method	RefCOCO	RefCOCO+	RefCOCOg
<i>Fine-tuned</i>			
LAVT [51]	72.7	62.1	61.2
CRIS [52]	70.5	65.3	59.9
PolyFormer-L [53]	76.94	72.15	71.15
SEEM [54]	—	—	65.7
LISA(SAM) [49]	74.9	65.1	67.9
PixelLM [55]	73.0	66.3	69.3
PerceptionGPT [56]	75.1	68.5	70.3
GSVA(SAM) [57]	77.2	65.9	72.7
SAM4MLLM(Qwen) [58]	77.1	71.5	74.5
SegAgent-Qwen+SAM [8]	78.01	70.86	74.49
<i>Zero-shot</i>			
SAM3+text [3]	39.40	29.44	32.76
A2A-GroundingModel	47.34	37.28	48.66
A2A-AffordGen	75.86	64.78	72.55

4.4 A2A-Policy: Simulation and Real-robot Manipulation

Controlled policy comparison. All four policies share the Diffusion Policy head [9] and Transformer fusion of visual, language, and state tokens. DP-RGB uses frozen DINOv2 without affordances. UAD-DP overlays UAD heatmaps on the RGB observation. A2A-Explicit overlays A2A-GroundingModel masks before RGB feature extraction. A2A-Implicit instead uses A2A-GroundingModel features with Afford-VLA-style [41] top-1 selection, thresholding, and mask pooling. Both A2A variants use the same frozen A2A-GroundingModel in simulation and on the robot; A2A-AffordGen does not enter the policy loop.

The implicit interface replaces unfiltered coarse FPN pooling while keeping the training setup unchanged. It restricts spatial aggregation to a predicted task-relevant region instead of asking the policy to select useful features from the entire coarse feature map. The comparison therefore tests two interfaces to the same grounding prior: explicit observation highlighting and implicit region-conditioned feature pooling.

Table 5. Policy evaluation with matched action-head and fusion architectures. LIBERO-Object reports the mean success rate over ten tasks; the real robot reports successes across four tasks with ten trials each. Higher is better.

Policy	LIBERO-Object (%)	Piper arm (successes/trials)
DP-RGB [9]	87.8	22/40
UAD-DP [6]	74.4	16/40
A2A-Explicit	94.4	26/40
A2A-Implicit	92.8	25/40

LIBERO-Object aggregate results. A2A-Explicit and A2A-Implicit achieve 94.4% and 92.8% mean success, compared with 87.8% for DP-RGB and 74.4% for UAD-DP (Tab. 5). Their absolute improvements over DP-RGB are 6.6 and 5.0 percentage points. The corresponding reductions in residual failure rate are

$$\frac{94.4 - 87.8}{100 - 87.8} \approx 54.1\%, \quad \frac{92.8 - 87.8}{100 - 87.8} \approx 41.0\%. \quad (13)$$

These are derived from the displayed aggregate success rates, not averages of per-task or per-seed relative reductions. Both interfaces improve on the RGB baseline, while explicit highlighting retains the strongest aggregate performance.

Object-wise variation. Explicit conditioning improves cream cheese from 90% to 98%, barbecue sauce from 72% to 86%, and chocolate pudding from 88% to 100%; milk remains at 92%. App. F provides the available per-task scores, whose means match Tab. 5. Only the aggregate is available for the updated implicit policy.

Real-robot results. A2A-Explicit achieves 26/40 successful trials and A2A-Implicit 25/40, compared with 22/40 for DP-RGB and 16/40 for UAD-DP. These totals correspond to 65.0%, 62.5%, 55.0%, and 40.0%, respectively. Explicit highlighting adds four successes over DP-RGB, while implicit pooling adds three. The per-task counts for the RGB and explicit-conditioning policies appear in App. F.

Explicit highlighting succeeds in nine microwave-opening, eight stacking, seven cup-placement, and two phone-placement trials, out of ten for each task. The corresponding DP-RGB counts are eight, six, six, and two. Phone placement remains difficult, with no improvement in these observed counts. A better functional-region prior can improve target selection without resolving contact, pose alignment, and action-execution difficulty. With ten trials per task, the individual counts are descriptive outcomes of this evaluation; we do not infer statistical significance from these totals alone.

Qualitative execution sequences. Figs. 8–11 in App. G show the four real-world demonstrations. Each figure follows the same row order, placing UAD-DP above A2A-Implicit and A2A-Explicit, with time advancing from left to right. The comparisons reveal how an affordance signal enters the policy observation or representation during execution. These selected sequences illustrate behavior and do not replace the complete trial counts.

Real-world distribution shifts can weaken UAD heatmaps or highlight an incorrect part, making an overlay misleading for the action model. Task-aligned masks provide a more targeted observation prior, while thresholded region pooling makes the implicit interface competitive without an overlay. Nevertheless, the unchanged phone-placement count and remaining failures in the other tasks show that grounding accuracy alone does not guarantee successful manipulation.

5 Conclusion and Limitation

Affordance2Action (A2A) connects scalable affordance annotation, real-time scene-level grounding, and robotic manipulation. **A2A-AffordGen** reduces annotation effort through high-quality masks and lightweight verification; **A2A-GroundingModel** improves task-conditioned localization at 9.7 FPS. With frozen grounding, **A2A-Policy**’s explicit variant raises success from 87.8% to 94.4% on LIBERO-Object and from 22/40 to 26/40 on the real robot. Its implicit variant reaches 92.8% and 25/40 with thresholded region pooling, showing that both tested interfaces benefit from the frozen grounding prior. The 200-scene multi-instance evaluation, component ablations, and human-verification audit jointly characterize annotation accuracy, instance coverage, and review effort. Despite these gains, broader deployment requires addressing the following limitations. Policy evaluation covers simulation and real Piper-arm tabletop tasks. Mobile and whole-body manipulation, including navigation, base placement, and coordinated arm execution in larger scenes, remain untested. Robot-induced occlusion, object motion, and contact disturbances can reduce affordance reliability. Beyond the current threshold-based filtering, dynamic grounding for closed-loop interaction remains future work.

References

- [1] Jinming Li, Yichen Zhu, Zhibin Tang, Junjie Wen, Minjie Zhu, Xiaoyu Liu, Chengmeng Li, Ran Cheng, Yaxin Peng, Yan Peng, et al. Coa-vla: Improving vision-language-action models via visual-text chain-of-affordance. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 9759–9769, 2025.
- [2] Jingjing Li, Yue Feng, Yuchen Guo, Jincai Huang, Wei Ji, Qi Bi, Yongri Piao, Miao Zhang, Xiaoqi Zhao, Qiang Chen, et al. Sam3-i: Segment anything with instructions. *arXiv preprint arXiv:2512.04585*, 2025.
- [3] Nicolas Carion, Laura Gustafson, Yuan-Ting Hu, Shoubhik Debnath, Ronghang Hu, Didac Suris, Chaitanya Ryali, Kalyan Vasudev Alwala, Haitham Khedr, Andrew Huang, et al. Sam 3: Segment anything with concepts. *arXiv preprint arXiv:2511.16719*, 2025.
- [4] Dongming Wu, Yanping Fu, Saikhe Huang, Yingfei Liu, Fan Jia, Nian Liu, Feng Dai, Tiancai Wang, Rao Muhammad Anwer, Fahad Shahbaz Khan, et al. Ragnet: Large-scale reasoning-based affordance segmentation benchmark towards general grasping. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 11980–11990, 2025.
- [5] Zifu Wan, Yaqi Xie, Ce Zhang, Zhiqiu Lin, Zihan Wang, Simon Stepputtis, Deva Ramanan, and Katia P Sycara. Instructpart: Task-oriented part segmentation with instruction reasoning. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 24202–24227, 2025.
- [6] Yihe Tang, Wenlong Huang, Yingke Wang, Chengshu Li, Roy Yuan, Ruohan Zhang, Jiajun Wu, and Li Fei-Fei. Uad: Unsupervised affordance distillation for generalization in robotic manipulation. In *2025 IEEE International Conference on Robotics and Automation (ICRA)*, pages 3822–3831. IEEE, 2025.
- [7] Shuai Shao, Zeming Li, Tianyuan Zhang, Chao Peng, Gang Yu, Xiangyu Zhang, Jing Li, and Jian Sun. Objects365: A large-scale, high-quality dataset for object detection. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 8430–8439, 2019.
- [8] Muzhi Zhu, Yuzhuo Tian, Hao Chen, Chunluan Zhou, Qingpei Guo, Yang Liu, Ming Yang, and Chunhua Shen. Segagent: Exploring pixel understanding capabilities in mllms by imitating human annotator trajectories. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pages 3686–3696, 2025.
- [9] Cheng Chi, Zhenjia Xu, Siyuan Feng, Eric Cousineau, Yilun Du, Benjamin Burchfiel, Russ Tedrake, and Shuran Song. Diffusion policy: Visuomotor policy learning via action diffusion. *The International Journal of Robotics Research*, 44(10-11):1684–1704, 2025.
- [10] Gen Li, Varun Jampani, Deqing Sun, and Laura Sevilla-Lara. Locate: Localize and transfer object parts for weakly supervised affordance grounding. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 10922–10931, 2023.
- [11] Gen Li, Deqing Sun, Laura Sevilla-Lara, and Varun Jampani. One-shot open affordance learning with foundation models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 3086–3096, 2024.
- [12] Hongchen Luo, Wei Zhai, Jing Zhang, Yang Cao, and Dacheng Tao. Learning affordance grounding from exocentric images. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 2252–2261, 2022.
- [13] Yuhang Yang, Wei Zhai, Hongchen Luo, Yang Cao, Jiebo Luo, and Zheng-Jun Zha. Grounding 3d object affordance from 2d interactions in images. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 10905–10915, 2023.
- [14] Shengheng Deng, Xun Xu, Chaozheng Wu, Ke Chen, and Kui Jia. 3d affordancenet: A benchmark for visual object affordance understanding. In *proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 1778–1787, 2021.
- [15] Hanqing Wang, Mingyu Liu, Xiaoyu Chen, Chengwei Ma, Yiming Zhong, Wenti Yin, Yuhao Liu, Zhiqing Cui, Jiahao Yuan, Lu Dai, et al. Videoafford: Grounding 3d affordance from human-object-interaction videos via multimodal large language model. *arXiv preprint arXiv:2602.09638*, 2026.
- [16] Shikhar Bahl, Russell Mendonca, Lili Chen, Unnat Jain, and Deepak Pathak. Affordances from human videos as a versatile representation for robotics. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 13778–13790, 2023.
- [17] Chunlin Yu, Hanqing Wang, Ye Shi, Haoyang Luo, Sibe Yang, Jingyi Yu, and Jingya Wang. Seqafford: Sequential 3d affordance reasoning via multimodal large language model. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 1691–1701, 2025.

- [18] Zixin Zhang, Kanghao Chen, Hanqing Wang, Hongfei Zhang, Harold Haodong Chen, Chenfei Liao, Litao Guo, and Ying-Cong Chen. A4-agent: An agentic framework for zero-shot affordance reasoning. *arXiv preprint arXiv:2512.14442*, 2025.
- [19] Shengyi Qian, Weifeng Chen, Min Bai, Xiong Zhou, Zhuowen Tu, and Li Erran Li. Affordancellm: Grounding affordance from vision language models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 7587–7597, 2024.
- [20] Zhenhao Zhang, Ye Shi, Lingxiao Yang, Suting Ni, Qi Ye, and Jingya Wang. Openhoi: Open-world hand-object interaction synthesis with multimodal large language model. *Advances in Neural Information Processing Systems*, 38:166582–166612, 2026.
- [21] Hanqing Wang, Shaoyang Wang, Yiming Zhong, Zemin Yang, Jiamin Wang, Zhiqing Cui, Jiahao Yuan, Yifan Han, Mingyu Liu, and Yuexin Ma. Affordance-r1: Reinforcement learning for generalizable affordance reasoning in multimodal large language models. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 40, pages 9738–9746, 2026.
- [22] Litao Liu, Wentao Wang, Yifan Han, Zhuoli Xie, Pengfei Yi, Junyan Li, and Wenzhao Lian. FoAM: Foresight-augmented multi-task imitation policy for robotic manipulation. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 40, pages 18460–18468, 2026. doi:10.1609/aaai.v40i22.38911.
- [23] Sixu Lin, Junliang Chen, Huaiyuan Xu, Zhuohao Li, Guangming Wang, Yixiong Jing, Sheng Xu, Runyi Zhao, Brian Sheil, Lap-Pui Chau, and Guiliang Liu. RoboFlow4D: A Lightweight Flow World Model Toward Real-Time Flow-Guided Robotic Manipulation. *arXiv preprint arXiv:2605.17522*, 2026.
- [24] Sixu Lin, Yunpeng Qing, Litao Liu, Ming Zhou, Ruixing Jin, Xiaoyi Fan, and Guiliang Liu. DyGRO-VLA: Cross-Task Scaling of Vision-Language-Action Models via Dynamic Grouped Residual Optimization. *arXiv preprint arXiv:2605.17486*, 2026.
- [25] Yunpeng Qing, Yixiao Chi, Shuo Chen, Shunyu Liu, Kexuan Zhou, Sixu Lin, Litao Liu, and Changqing Zou. BiTrajDiff: Bidirectional trajectory generation with diffusion models for offline reinforcement learning. *arXiv preprint arXiv:2506.05762*, 2025.
- [26] Qin Liu, Zhenlin Xu, Gedas Bertasius, and Marc Niethammer. Simpleclick: Interactive image segmentation with simple vision transformers. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 22290–22300, 2023.
- [27] Ning Xu, Brian Price, Scott Cohen, Jimei Yang, and Thomas S Huang. Deep interactive object selection. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 373–381, 2016.
- [28] Qwen Team. Qwen3.5-9B. Model card, 2026. URL <https://huggingface.co/Qwen/Qwen3.5-9B>.
- [29] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. *arXiv:2106.09685*, 2021.
- [30] Shilong Liu, Zhaoyang Zeng, Tianhe Ren, Feng Li, Hao Zhang, Jie Yang, Qing Jiang, Chunyuan Li, Jianwei Yang, Hang Su, et al. Grounding dino: Marrying dino with grounded pre-training for open-set object detection. In *European conference on computer vision*, pages 38–55. Springer, 2024.
- [31] Robert M Haralick, Stanley R Sternberg, and Xinhua Zhuang. Image analysis using mathematical morphology. *IEEE transactions on pattern analysis and machine intelligence*, (4):532–550, 1987.
- [32] Shuai Bai, Yuxuan Cai, Ruizhe Chen, Keqin Chen, Xionghui Chen, Zesen Cheng, Lianghao Deng, Wei Ding, Chang Gao, Chunjiang Ge, Wenbin Ge, Zhifang Guo, Qidong Huang, Jie Huang, Fei Huang, Binyuan Hui, Shutong Jiang, Zhaohai Li, Mingsheng Li, Mei Li, Kaixin Li, Zicheng Lin, Junyang Lin, Xuejing Liu, Jiawei Liu, Chenglong Liu, Yang Liu, Dayiheng Liu, Shixuan Liu, Dunjie Lu, Ruilin Luo, Chenxu Lv, Rui Men, Lingchen Meng, Xuancheng Ren, Xingzhang Ren, Sibao Song, Yuchong Sun, Jun Tang, Jianhong Tu, Jianqiang Wan, Peng Wang, Pengfei Wang, Qiuyue Wang, Yuxuan Wang, Tianbao Xie, Yiheng Xu, Haiyang Xu, Jin Xu, Zhibo Yang, Mingkun Yang, Jianxin Yang, An Yang, Bowen Yu, Fei Zhang, Hang Zhang, Xi Zhang, Bo Zheng, Humen Zhong, Jingren Zhou, Fan Zhou, Jing Zhou, Yuanzhi Zhu, and Ke Zhu. Qwen3-VL technical report. *arXiv:2511.21631*, 2025.
- [33] Andrew Guo, Bowen Wen, Jianhe Yuan, Jonathan Tremblay, Stephen Tyree, Jeffrey Smith, and Stan Birchfield. Handal: A dataset of real-world manipulable object categories with pose annotations, affordances, and reconstructions. In *2023 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 11428–11435. IEEE, 2023.
- [34] Zifu Wan, Yaqi Xie, Ce Zhang, Zhiqiu Lin, Zihan Wang, Simon Stepputtis, Deva Ramanan, and Katia P Sycara. Instructpart: Affordance-based part segmentation from language instruction. In *AAAI-2024 Workshop on Public Sector LLMs: Algorithmic and Sociotechnical Design*, 2024.

- [35] Hao-Shu Fang, Chenxi Wang, Minghao Gou, and Cewu Lu. Graspnet-1billion: A large-scale benchmark for general object grasping. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 11444–11453, 2020.
- [36] Abby O’Neill, Abdul Rehman, Abhiram Maddukuri, Abhishek Gupta, Abhishek Padalkar, Abraham Lee, Acorn Pooley, Agrim Gupta, Ajay Mandlekar, Ajinkya Jain, et al. Open x-embodiment: Robotic learning datasets and rt-x models: Open x-embodiment collaboration 0. In *2024 IEEE International Conference on Robotics and Automation (ICRA)*, pages 6892–6903. IEEE, 2024.
- [37] Chenchen Zhu, Fanyi Xiao, Andrés Alvarado, Yasmine Babaei, Jiabo Hu, Hichem El-Mohri, Sean Culatana, Roshan Sumbaly, and Zhicheng Yan. Egoobjects: A large-scale egocentric dataset for fine-grained object understanding. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 20110–20120, 2023.
- [38] Shengyi Qian and David F Fouhey. Understanding 3d object interaction from a single image. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 21753–21763, 2023.
- [39] Tony Z Zhao, Vikash Kumar, Sergey Levine, and Chelsea Finn. Learning fine-grained bimanual manipulation with low-cost hardware. *arXiv preprint arXiv:2304.13705*, 2023.
- [40] Haoquan Fang, Markus Grotz, Wilbert Pumacay, Yi Ru Wang, Dieter Fox, Ranjay Krishna, and Jiafei Duan. Sam2act: Integrating visual foundation model with a memory architecture for robotic manipulation. *arXiv preprint arXiv:2501.18564*, 2025.
- [41] Runze Wang, Yuqian Fu, Yu Li, Tao Lin, Tianwen Qian, Mohamed Elhoseiny, Bo Zhao, Yanwei Fu, Yungang Jiang, and Xiangyang Xue. Afford-vla: Action-aligned visual planning via internalized affordance. *arXiv:2605.24203*, 2026.
- [42] Licheng Yu, Patrick Poirson, Shan Yang, Alexander C Berg, and Tamara L Berg. Modeling context in referring expressions. In *European conference on computer vision*, pages 69–85. Springer, 2016.
- [43] Sahar Kazemzadeh, Vicente Ordonez, Mark Matten, and Tamara Berg. Referitgame: Referring to objects in photographs of natural scenes. In *Proceedings of the 2014 conference on empirical methods in natural language processing (EMNLP)*, pages 787–798, 2014.
- [44] Junhua Mao, Jonathan Huang, Alexander Toshev, Oana Camburu, Alan L Yuille, and Kevin Murphy. Generation and comprehension of unambiguous object descriptions. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 11–20, 2016.
- [45] Bo Liu, Yifeng Zhu, Chongkai Gao, Yihao Feng, Qiang Liu, Yuke Zhu, and Peter Stone. Libero: Benchmarking knowledge transfer for lifelong robot learning. *Advances in Neural Information Processing Systems*, 36:44776–44791, 2023.
- [46] Maxime Oquab, Timothée Darcet, Théo Moutakanni, Huy Vo, Marc Szafraniec, Vasil Khalidov, Pierre Fernandez, Daniel Haziza, Francisco Massa, Alaaeldin El-Nouby, et al. Dinov2: Learning robust visual features without supervision. *arXiv preprint arXiv:2304.07193*, 2023.
- [47] Jinze Bai, Shuai Bai, Yunfei Chu, Zeyu Cui, Kai Dang, Xiaodong Deng, Yang Fan, Wenbin Ge, Yu Han, Fei Huang, et al. Qwen technical report. *arXiv preprint arXiv:2309.16609*, 2023.
- [48] Alexander Kirillov, Eric Mintun, Nikhila Ravi, Hanzi Mao, Chloe Rolland, Laura Gustafson, Tete Xiao, Spencer Whitehead, Alexander C Berg, Wan-Yen Lo, et al. Segment anything. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 4015–4026, 2023.
- [49] Xin Lai, Zhuotao Tian, Yukang Chen, Yanwei Li, Yuhui Yuan, Shu Liu, and Jiaya Jia. Lisa: Reasoning segmentation via large language model. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 9579–9589, 2024.
- [50] Zihan Zhong, Zhiqiang Tang, Tong He, Haoyang Fang, and Chun Yuan. Convolution meets lora: Parameter efficient finetuning for segment anything model. *arXiv preprint arXiv:2401.17868*, 2024.
- [51] Zhao Yang, Jiaqi Wang, Yansong Tang, Kai Chen, Hengshuang Zhao, and Philip HS Torr. Lavt: Language-aware vision transformer for referring image segmentation. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 18155–18165, 2022.
- [52] Zhaoqing Wang, Yu Lu, Qiang Li, Xunqiang Tao, Yandong Guo, Mingming Gong, and Tongliang Liu. Cris: Clip-driven referring image segmentation. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 11686–11695, 2022.
- [53] Jiang Liu, Hui Ding, Zhaowei Cai, Yuting Zhang, Ravi Kumar Satzoda, Vijay Mahadevan, and R Manmatha. Polyformer: Referring image segmentation as sequential polygon generation. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 18653–18663, 2023.

- [54] Xueyan Zou, Jianwei Yang, Hao Zhang, Feng Li, Linjie Li, Jianfeng Wang, Lijuan Wang, Jianfeng Gao, and Yong Jae Lee. Segment everything everywhere all at once. *Advances in neural information processing systems*, 36:19769–19782, 2023.
- [55] Zhongwei Ren, Zhicheng Huang, Yunchao Wei, Yao Zhao, Dongmei Fu, Jiashi Feng, and Xiaojie Jin. Pixellm: Pixel reasoning with large multimodal model. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 26374–26383, 2024.
- [56] Renjie Pi, Lewei Yao, Jiahui Gao, Jipeng Zhang, and Tong Zhang. Perceptiongpt: Effectively fusing visual perception into llm. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 27124–27133, 2024.
- [57] Zhuofan Xia, Dongchen Han, Yizeng Han, Xuran Pan, Shiji Song, and Gao Huang. Gsva: Generalized segmentation via multimodal large language models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 3858–3869, 2024.
- [58] Yi-Chia Chen, Wei-Hua Li, Cheng Sun, Yu-Chiang Frank Wang, and Chu-Song Chen. Sam4mllm: Enhance multi-modal large language model for referring expression segmentation. In *European Conference on Computer Vision*, pages 323–340. Springer, 2024.

A A2A-AffordGen: Training and Inference Details

This section expands the construction and inference procedures in Sec. 3.1. The learned component is the click policy; SAM3 supplies the mask transition, and MAG supplies the scene-level wrapper. These components have different responsibilities and do not share a single end-to-end training loss.

A.1 Supervision and Serialized Click Targets

The training corpus combines approximately 40K curated object-part masks with approximately 18K manually annotated part masks from the 5,000-scene core, giving approximately 58K object-part masks before trajectory expansion. The oracle converts these masks into approximately 150K click-trajectory samples. The 5,000 automatically generated, human-verified scenes form the initial released batch; they are distinct from the manually annotated core and from the held-out evaluation scenes. A scene may contribute several functional regions and therefore several crop-level training examples.

For a crop I , explicit part prompt P , and current mask M_t , the policy state contains the image with its current-mask overlay and the prompt. The reference mask M^* is used to produce supervision. The two residual sets,

$$E_t^+ = M^* \setminus M_t, \quad E_t^- = M_t \setminus M^*, \quad (14)$$

separate missed target pixels from incorrectly included pixels. The oracle chooses the larger residual set and places a click at its distance-transform maximum, as in Eq. 1. Positive and negative clicks therefore have distinct roles even when they occur on the same object. After each oracle action, frozen SAM3 materializes the next mask; the next training state reflects that updated mask rather than reusing the initial prediction.

The target sequence contains four fields: click polarity, click location, next-mask IoU, and incremental IoU gain. One example of the structured output is “*Positive point: (x, y). Next IoU: v. Gain: δ .*” The quality targets are measured after applying the oracle click:

$$v_t^* = \text{IoU}(M_{t+1}^*, M^*), \quad \delta_t^* = v_t^* - \text{IoU}(M_t, M^*), \quad (15)$$

where $M_{t+1}^* = \Phi(I, P, M_t, a_t^*)$ denotes the result of the oracle action, not a second ground-truth mask. This distinguishes the target quality of the proposed next state from the quality of the current state.

Qwen3.5-9B is adapted with LoRA to predict the serialized sequence y_t^* . Writing its tokens as $y_{t,1}^*, \dots, y_{t,L_t}^*$, the token-level cross-entropy in Eq. 3 has the autoregressive form

$$\mathcal{L}_{\text{agent}}(s_t, y_t^*) = - \sum_{j=1}^{L_t} \log p_{\theta}(y_{t,j}^* \mid s_t, y_{t,<j}^*). \quad (16)$$

Thus, the click and stopping estimates are learned through the same structured text target. No pixel-level segmentation loss is back-propagated through SAM3 into the language model. Oracle training rollouts terminate at the reference-IoU target κ or the click budget. At deployment, the reference mask and oracle residual sets are unavailable.

A.2 Stopping Order and Seeded Refinement

SAG can start from an empty mask or from a low-confidence SAM3 mask. In both cases, the same trained policy receives the current mask and proposes a correction. Alg. 1 uses an in-place mask update and permits at most eight clicks. The order matters because the predicted quality describes the next mask:

1. Query the policy for $(p_t, u_t, \hat{v}_t, \hat{\delta}_t)$ using the current mask.
2. If $\hat{\delta}_t < \Delta_{\min}$, stop and return the current mask without applying the proposed click.
3. Otherwise, apply the click through frozen SAM3 and replace the current mask with the result.
4. If $\hat{v}_t > \tau_{\text{IoU}}$, stop with that updated mask; otherwise continue until the click budget is exhausted.

The reference-IoU target used to construct training rollouts is separate from these predicted stopping signals. A seed mask also changes the starting state, not the objective or policy parameters. Removing the seed in the ablation forces refinement to reconstruct the region from an empty mask and costs 4.3 gIoU points in the full pipeline.

A.3 MAG Crops, Mask-out, and Scene Reassembly

SAM3 text prompting returns candidate masks and confidence scores, while Grounding DINO supplies object boxes. A confident candidate bypasses SAG. A low-confidence candidate supplies both an initial mask and a matched crop for seeded refinement. An unmatched object box starts SAG from an empty mask, including when SAM3 has returned no mask for the scene. The matching and crop wrapper allow the single-instance policy to be reused without training a separate multi-instance click model.

For seeded refinement, let $D_i = m_i \oplus \mathcal{K}$ be the dilated candidate region and let g denote the neutral gray background. The image presented to the crop operator is

$$I_i^{\text{gray}}(u) = \begin{cases} I(u), & u \in D_i, \\ g, & u \notin D_i. \end{cases} \quad I_{b_i} = \mathcal{C}_{b_i}(I_i^{\text{gray}}). \quad (17)$$

The initial mask is transformed with the same crop operator, $M_0 = \mathcal{C}_{b_i}(m_i)$. Dilation preserves a local context band, while gray mask-out suppresses neighboring objects. An unmatched box has no seed mask, so its image is cropped directly from I and its rollout starts from $M_0 = \emptyset$.

Each refined crop mask is mapped back with $\mathcal{C}_{b_i}^{-1}$. Directly accepted masks already use scene coordinates and require the identity mapping. Reassembly retains the individual masks as an instance set, as in Eq. 4; it does not discard instance identities by collapsing all output regions into one binary target. This distinction matters for the padded set matching and count error in Sec. 4.1.

Automatic refinement is distinct from the subsequent human verification stage. MAG chooses whether to accept, refine, or initialize an instance; a human reviewer subsequently decides whether the resulting annotation can be retained, corrected, or rejected. App. D reports both processes with their respective denominators.

A.4 Training Behavior

Fig. 6 shows the training traces with matched A2A-Bench splits and optimizer settings. A2A-AffordGen learns a serialized click-policy target against frozen SAM3, whereas LISA-7B [49] optimizes its own segmentation objective. The curves therefore visualize optimization within each model rather than a common loss scale.

A2A-AffordGen shows a rapid initial decrease followed by a sustained plateau with limited fluctuations. LISA also decreases initially, then improves more gradually with step-to-step variation. These traces describe optimization behavior for the displayed runs. Grounding quality is established by the held-out mask and set metrics in Tab. 1, rather than by comparing absolute loss values or inferring cross-seed stability from one plot.

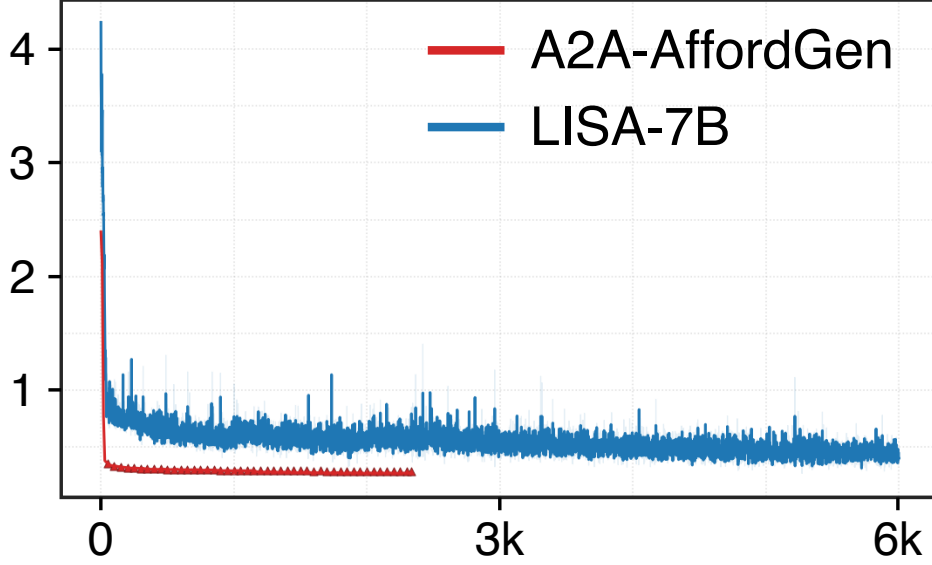


Figure 6. Training trends of A2A-AffordGen and LISA-7B. The horizontal axis is the training step; the vertical axis shows each model’s own training objective. Red triangles mark A2A-AffordGen validation probes. Different objectives prevent interpreting the absolute loss gap as a quality comparison.

B A2A-GroundingModel: Detailed Formulation

This section provides the full formulation of A2A-GroundingModel, including staged instruction adaptation, text-conditioned visual prompt injection, and the training objective. The main text gives the core equations; the step-by-step formulation below separates the query routes, prompt tensors, and training targets.

B.1 Staged Instruction Adaptation

A2A-GroundingModel uses two types of language supervision. Oracle queries explicitly describe the target functional part, while task queries describe the manipulation intent and require the model to infer the corresponding actionable region. To reduce the interference between explicit part recognition and task-level reasoning, we introduce separate but connected adaptation paths for oracle and task instructions. Routing is fixed by the supplied query type rather than selected by a learned gate. ORPS uses the O-Adapter route; TRPS uses the oracle-to-task bridge and T-Adapter route. Only the task route is used at deployment.

Let $\mathbf{x}^{(\ell)}$ denote the hidden state at the ℓ -th adapterized block. For oracle part descriptions, we use a lightweight bottleneck adapter $A_o^{(\ell)}$:

$$\mathbf{x}_o^{(\ell)} = \mathbf{x}^{(\ell)} + A_o^{(\ell)} \left(\text{LN}(\mathbf{x}^{(\ell)}) \right). \quad (18)$$

For task instructions, we introduce an oracle-to-task adaptation path. The hidden state is first transformed by an oracle-to-task bridge adapter $A_{o \rightarrow t}^{(\ell)}$, and then further refined by a task-specific adapter $A_t^{(\ell)}$:

$$\begin{aligned} \mathbf{r}^{(\ell)} &= \mathbf{x}^{(\ell)} + A_{o \rightarrow t}^{(\ell)} \left(\text{LN}(\mathbf{x}^{(\ell)}) \right), \\ \mathbf{x}_t^{(\ell)} &= \mathbf{r}^{(\ell)} + A_t^{(\ell)} \left(\text{LN}(\mathbf{r}^{(\ell)}) \right). \end{aligned} \quad (19)$$

This staged design allows the model to first acquire stable part-level grounding from explicit part queries and then transfer this knowledge to task-level affordance reasoning. We apply the same adaptation strategy to selected language, fusion, decoder, and mask prediction blocks.

B.2 Text-Conditioned Visual Prompt Injection

Staged instruction adaptation improves task understanding, but task-conditioned affordance grounding also requires accurate localization of small and visually subtle functional parts. To inject task information into the visual representation, we introduce a text-conditioned visual prompt generator.

Given token-level language features $\mathbf{T} \in \mathbb{R}^{L \times d_t}$, we obtain a task representation $\mathbf{z} \in \mathbb{R}^{d_t}$ by masked average pooling over valid text tokens. With $\mathcal{I}_q$ denoting the valid, non-padding token positions,

$$\mathbf{z} = \frac{1}{|\mathcal{I}_q|} \sum_{i \in \mathcal{I}_q} \mathbf{T}_i. \quad (20)$$

In parallel, the image is patch-projected and processed by a shallow visual projection network to produce spatial prompt tokens $\mathbf{V} \in \mathbb{R}^{HW \times d_p}$. The task representation is mapped to a text bias and a channel-wise gate:

$$\mathbf{b} = W_b \text{LN}(\mathbf{z}), \quad \mathbf{g} = \sigma(W_g \text{LN}(\mathbf{z})), \quad (21)$$

where $\mathbf{b}, \mathbf{g} \in \mathbb{R}^{d_p}$.

We then compute a text-guided spatial weighting over visual prompt tokens:

$$\alpha_j = \frac{\exp(\mathbf{V}_j^\top \mathbf{b} / \sqrt{d_p})}{\sum_k \exp(\mathbf{V}_k^\top \mathbf{b} / \sqrt{d_p})}, \quad (22)$$

where j indexes the spatial location.

The resulting text-conditioned prompt seed is defined as

$$\mathbf{S}_j = \text{LN}(\mathbf{V}_j \odot (1 + \alpha_j \mathbf{g}) + \mathbf{b}). \quad (23)$$

Layer-specific projections then transform $\mathbf{S}$ into prompt tensors $\mathbf{P}^{(\ell)}$ that match the channel dimension of selected visual encoder blocks. For each prompted layer ℓ , the visual tokens are updated through additive prompt injection:

$$\mathbf{X}^{(\ell)} \leftarrow \mathbf{X}^{(\ell)} + \gamma_\ell \mathbf{P}^{(\ell)}, \quad (24)$$

where γ_ℓ is a learnable scaling coefficient. We inject prompts into later visual encoder blocks, where visual features are more semantically structured and therefore more suitable for task-conditioned modulation.

B.3 Training Objective

During training, the base model is kept frozen and only the newly introduced adaptation modules are optimized. The model is trained with both oracle part queries and task-level instructions from A2A-Bench. Oracle queries provide explicit part-level supervision, while task queries require grounding the functional region from manipulation intent.

Predicted masks are matched to ground-truth masks by Hungarian assignment. The base segmentation objective is denoted as $\mathcal{L}_{\text{seg}}$, which includes classification, box regression, generalized-IoU, and mask supervision terms:

$$\mathcal{L}_{\text{seg}} = \lambda_{\text{cls}} \mathcal{L}_{\text{cls}} + \lambda_{\text{box}} \mathcal{L}_{\text{box}} + \lambda_{\text{giou}} \mathcal{L}_{\text{giou}} + \lambda_{\text{mask}} \mathcal{L}_{\text{mask}}. \quad (25)$$

To align explicit part grounding and task-level affordance grounding, we further impose oracle-task consistency regularization. Let $\mathbf{P}_o$ and $\mathbf{P}_t$ denote the spatial affordance distributions predicted from the oracle query and task query, respectively. We use a bidirectional KL divergence:

$$\mathcal{L}_{\text{KL}} = \frac{1}{2} [D_{\text{KL}}(\mathbf{P}_o \parallel \mathbf{P}_t) + D_{\text{KL}}(\mathbf{P}_t \parallel \mathbf{P}_o)]. \quad (26)$$

We also introduce a disagreement-weighted task-query loss to emphasize pixels where the oracle and task predictions are inconsistent. Let $\mathbf{W}_{\text{task}}$ be a disagreement weight map computed from the prediction difference between the two routes. The task-query loss is written as

$$\mathcal{L}_{\text{task}} = \frac{1}{|\Omega|} \sum_{u \in \Omega} \mathbf{W}_{\text{task}}(u) \cdot \mathcal{L}_{\text{mask}}(\hat{M}_{\text{task}}(u), M(u)), \quad (27)$$

where Ω denotes the image lattice, $\hat{M}_{\text{task}}$ is the task-query prediction, and M is the ground-truth affordance mask.

The final training objective is

$$\mathcal{L} = \mathcal{L}_{\text{seg}} + \lambda_{\text{KL}}\mathcal{L}_{\text{KL}} + \lambda_{\text{task}}\mathcal{L}_{\text{task}}. \quad (28)$$

This objective encourages the model to preserve accurate part-level localization from explicit oracle queries while producing spatially consistent predictions under task-level instructions.

Training schedule. We follow a three-stage oracle-to-task curriculum. In Stage I, the model is trained with oracle part queries to learn explicit part-level grounding. In Stage II, the task route is trained with task-level instructions to transfer this grounding ability to affordance reasoning. In Stage III, oracle and task routes are jointly optimized with an additional oracle–task consistency loss to align their spatial predictions. Throughout training, the base model is frozen and only the inserted adaptation modules are updated.

C UAD Adaptation and Original Diagnostic Experiments

C.1 Supervision Interface and Training Loss

UAD retains its heatmap architecture. To adapt its supervision to A2A-Bench, each image is paired with its task query, stored as `complex_query` in the original data interface. All positive instance masks associated with that query are merged by pixel-wise union into one binary target, which is resized to the UAD output grid. This creates a heatmap-supervision target without changing the UAD model into an instance-set predictor.

Let $p_j \in [0, 1]$ be the predicted affordance probability and $y_j \in [0, 1]$ the target at output location j . With M spatial locations, the binary cross-entropy is

$$\mathcal{L}_{\text{BCE}} = -\frac{1}{M} \sum_{j=1}^M [y_j \log p_j + (1 - y_j) \log(1 - p_j)]. \quad (29)$$

The soft Dice loss is

$$\mathcal{L}_{\text{Dice}} = 1 - \frac{2 \sum_j p_j y_j + \epsilon}{\sum_j p_j + \sum_j y_j + \epsilon}, \quad \epsilon = 1.0. \quad (30)$$

The original comparison includes a BCE-only variant and a variant trained with

$$\mathcal{L}_{\text{UAD}} = \mathcal{L}_{\text{BCE}} + \lambda \mathcal{L}_{\text{Dice}}, \quad \lambda = 1.0. \quad (31)$$

For BCE+Dice, target-mask blur is disabled to match the binary annotations. Tab. 6 lists the training configuration. The output grid is smaller than the input image; target projection and heatmap thresholding therefore form part of the evaluation interface.

Table 6. Training settings for the adapted UAD baseline on A2A-Bench.

Item	Setting
Input resolution	448×448
Output resolution	32×32
Optimizer	Adam
Learning rate	2×10^{-3}
Batch size	128
Training epochs	50
LR schedule	Step decay every 10 epochs with factor 0.75

C.2 BCE versus BCE+Dice

The following two tables preserve the diagnostic experiments reported in the first manuscript. Their evaluation manifest and seed aggregation are not specified in the available record, so these results are reported separately from the current 200-pair, three-seed TRPS benchmark in Tab. 3. They

explain the original UAD adaptation and threshold choice; they are not additional matched runs on the current benchmark.

Tab. 7 compares predictions thresholded at 0.5 with the projected target mask. Here 0.5 is the heatmap binarization threshold, not the per-example IoU threshold used by P@50. The BCE and Dice-loss columns evaluate soft predictions; the remaining columns evaluate binary masks. Consequently, one minus the displayed soft Dice loss need not equal the displayed binary Dice score.

Table 7. Original UAD supervision comparison at heatmap threshold 0.5. BCE and Dice loss use soft predictions; binary overlap, precision, and recall are percentages. These diagnostic results are separate from the current 200-pair benchmark.

Variant	BCE ↓	Dice Loss ↓	IoU ↑	Dice ↑	Precision ↑	Recall ↑
UAD-BCE	0.1015	0.7634	13.92	18.55	76.55	17.14
UAD-BCE+Dice	0.1224	0.6749	25.76	33.40	60.93	34.99

BCE-only training has lower BCE loss and higher binary precision, but recovers a smaller fraction of the target pixels. With BCE+Dice, IoU increases from 13.92% to 25.76%, Dice from 18.55% to 33.40%, and recall from 17.14% to 34.99%, while precision decreases from 76.55% to 60.93%. This is a coverage–precision tradeoff rather than an improvement in every metric. The BCE objective’s lower numerical value does not by itself establish a better binary mask, and the soft-loss columns should not be compared directly with the percentage-valued overlap columns.

C.3 Heatmap Threshold Sensitivity

Tab. 8 reports the original sweep for BCE+Dice without target blur. Among the displayed settings, 0.1 gives the highest gIoU and P@50, 0.2 the highest cIoU, and 0.3 the highest P@50–95. The metric-specific optima show that selecting a threshold for mean mask overlap need not maximize pooled overlap or success at stricter IoU cutoffs. The mean-overlap score falls from 28.45% at threshold 0.1 to 15.77% at threshold 0.9, consistent with increasing suppression of heatmap responses.

Table 8. Original UAD threshold diagnostic with BCE+Dice supervision and no target blur. Scores are percentages; the original values are retained separately from the current 200-pair, three-seed results.

Threshold	gIoU (%)	cIoU (%)	P@50 (%)	P@50-95 (%)
0.1	28.45	36.39	28.22	10.03
0.2	28.23	36.71	27.18	10.77
0.3	27.70	36.43	26.83	11.01
0.5	25.76	34.66	24.74	10.52
0.7	22.27	31.33	18.82	8.71
0.9	15.77	23.96	14.63	5.44

The current grounding evaluation uses threshold 0.1, while the downstream UAD-DP overlay uses 0.7. These thresholds serve different purposes. The original threshold-0.1 row reports P@50 of 28.22%; the current 200-pair benchmark reports 28.17%. We retain each table’s own recorded value rather than combining the two evaluations. Likewise, the 25.76% IoU in the original BCE+Dice comparison agrees with the threshold-0.5 sweep row, providing a cross-check within those original diagnostics.

D Human Verification and Annotation Routing

Human verification and correction. We audit 500 newly generated single-instance Objects365 masks and 500 newly generated multi-instance masks. Reviewers use per-part click correction, threshold adjustment, accept/reject decisions, and missed-region counting. These 500-mask audits assess annotation usability and are separate from the 200-scene grounding benchmarks. Tab. 9 reports the decisions and time cost.

For SAG, 76.6% of masks are accepted directly, 21.2% are corrected, and 2.2% are rejected, with 4.5 seconds of review per mask. MAG requires more intervention: 65.4% are accepted, 30.2%

Table 9. Human verification of newly generated Objects365 masks. SAG and MAG each contribute 500 masks to this audit. Decision rates are percentages; review time is seconds per mask. From-scratch annotation takes approximately 90 seconds per mask.

Regime	Masks	Accept (%)	Correct (%)	Reject (%)	Review (s)
SAG	500	76.6	21.2	2.2	4.5
MAG	500	65.4	30.2	4.4	10.4

corrected, and 4.4% rejected, with 10.4 seconds of review per mask. Compared with approximately 90 seconds for from-scratch annotation, these timings indicate a substantial reduction in manual effort through verification and targeted edits. They describe human review time, not end-to-end generation latency or robot control frequency.

Table 10. MAG branch usage. Percentages are computed over all target instances processed by MAG, including instances without a matched SAM3 mask. They sum to 100%.

Branch	Operation	Target instances (%)
Direct acceptance	Retain a confident SAM3 mask	53.0
SAG-refine	Refine a low-confidence seed mask	13.8
SAG-from-scratch	Initialize from an empty mask	33.2

Tab. 10 reports how the pipeline allocates refinement. Direct acceptance handles 53.0% of target instances, SAG-refine handles 13.8%, and SAG-from-scratch handles 33.2%. The denominator includes all target instances, including those for which SAM3 supplies no matched mask. Thus, the two iterative branches together handle 47.0% of instances. The multi-instance audit recall is 78.2% over valid regions, counting corresponding masks regardless of their quality. This coverage-oriented audit differs from Set-R, which requires one-to-one matching at IoU at least 0.5; the two recall values should not be interpreted as the same metric.

What refinement contributes. Direct acceptance preserves confident SAM3 predictions without invoking the click policy. SAG-refine corrects a supplied candidate while retaining its useful initial geometry; SAG-from-scratch provides a path for targets without an initial mask. These branch frequencies measure how often each operation is used, not the accuracy gain of individual corrected masks. The component ablations in Tab. 2 provide the corresponding pipeline-level evidence: disabling iterative clicks costs 10.4 gIoU points, removing the direct-accept branch costs 4.7, and discarding the refinement seed costs 4.3. Removing dilated gray mask-out costs 2.5 points, and reducing the click cap from eight to two costs 3.2. These are separate ablations of the full system, so their effects are not added together.

Automatic refinement versus human correction. The 13.8% SAG-refine share describes automatic inference before human review; the 30.2% MAG correction rate describes a later human decision over reviewed output masks. Their denominators and stages differ, so they do not form a joint breakdown. The same distinction separates automatic direct acceptance from a reviewer’s final acceptance. A mask can enter the review tool after any automatic branch. Rejected masks can subsequently be manually reannotated for additional supervision, but this possibility does not change the reported accept/correct/reject audit outcomes.

Verification interface. Fig. 7 shows the original image, mask overlay, and mask-only views used to inspect part annotations. The interface exposes query text, per-part thresholds, automatic branch and click information, missed-instance counts, and accept/reject controls. Reviewers can distinguish wrong objects or parts from over-segmentation, under-segmentation, and noisy boundaries. Per-part click correction and threshold adjustment provide targeted edits, while missed-region counting records omissions that would be invisible if reviewers inspected only the proposed masks. The threshold control displayed in this interface is a review tool setting, not the UAD benchmark threshold.

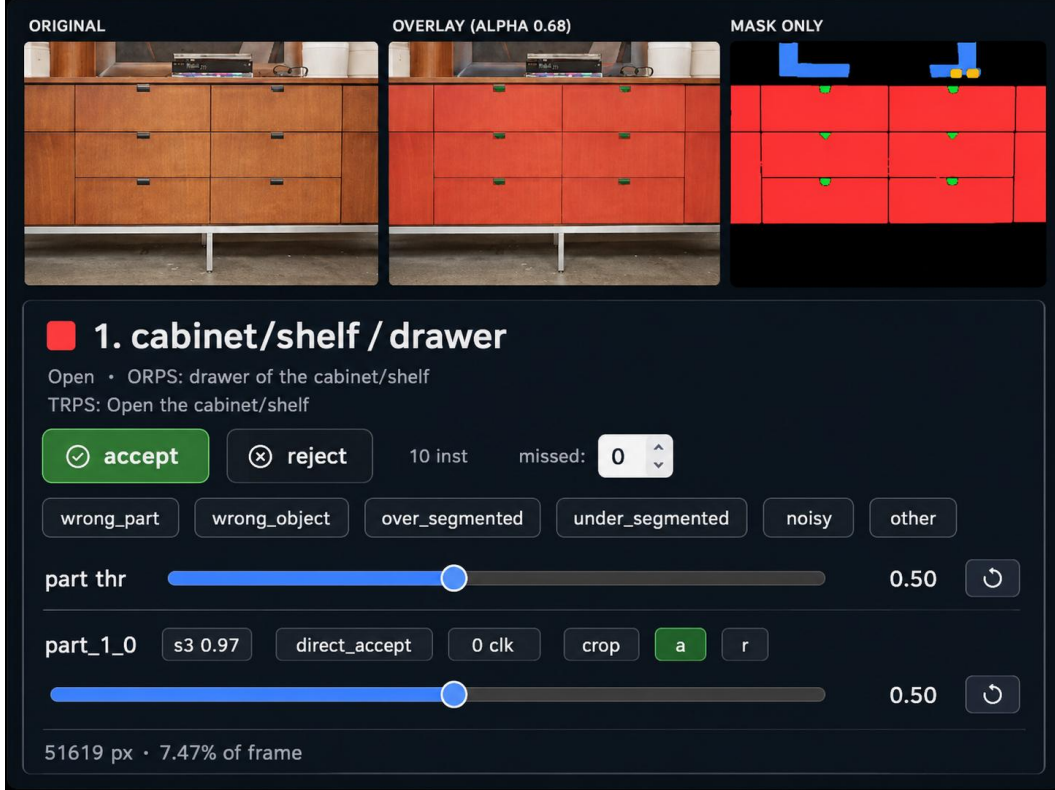


Figure 7. Human verification interface. Original, overlay, and mask-only views accompany part-level decisions, threshold controls, branch metadata, and missed-instance counting. The displayed example illustrates the tool; audit statistics are reported in Tab. 9.

E A2A-Policy: Architecture, Training, and Deployment

The comparison changes the affordance interface while keeping the action head and multimodal fusion architecture shared. Both A2A variants use the same frozen A2A-GroundingModel checkpoint in simulation and on the robot. A2A-AffordGen is used for annotation and does not supply masks or features inside the policy loop. Grounding-model parameters remain frozen while robot demonstrations train the policy modules.

E.1 Visual and Language Interfaces

DP-RGB. The RGB baseline encodes raw observations with frozen DINOv2-ViT-L/14 features. It receives no predicted affordance overlay. Its Transformer fusion over image, language, and state tokens makes it a strong RGB reference rather than the original 1D-UNet Diffusion Policy architecture.

UAD-DP. The UAD heatmap is thresholded at 0.7 and overlaid on RGB before the same DINOv2 encoder. Overlay observations can be precomputed for the training replay buffer. The policy-overlay threshold is separate from the threshold used to evaluate UAD segmentation accuracy.

A2A-Explicit. A2A-GroundingModel predicts the task-relevant region. The highest-confidence region is thresholded and rendered as a colored alpha overlay before DINOv2 feature extraction. During training, these overlays can be precomputed from the frozen model; at execution, the grounding input is the current observation and instruction.

A2A-Implicit. The same frozen grounding checkpoint supplies multi-scale FPN features. Following the updated interface, the highest-confidence predicted region is selected and thresholded before spatial mask pooling. The region-conditioned features are projected to the policy width and fused with instruction and state tokens. Only the $\langle \text{ACT} \rangle$ token conditions the diffusion head. This is

the interface associated with the current 92.8% LIBERO and 25/40 real-robot aggregates; unfiltered coarse pooling is not used for these results.

E.2 Shared Architecture and Optimization

Tab. 11 records the shared architecture and training recipe. The action head predicts a normalized 16-step action chunk from two observation frames, with seven values per action. All four variants train 55.8M parameters in the policy modules while leaving their visual backbones frozen. The current configuration uses per-GPU batch size 16 for all four variants.

Table 11. A2A-Policy architecture and LIBERO training settings. The visual/affordance interface differs between policies; fusion and action prediction are shared.

Setting	Value
Observation history / action horizon	2 frames / 16 steps
Action dimension	7
Fusion layers / width	4 / 1024
Attention heads / feed-forward width	8 / 2048
Diffusion-head depth / width	6 / 768
Trainable policy parameters	55.8M
Training diffusion steps	100
Optimizer / learning rate	AdamW / 10^{-4}
Training epochs / per-GPU batch size	30 / 16
Warm-up / schedule	500 steps / cosine decay
Weight decay / gradient clipping	10^{-2} / 3.0
Precision / validation interval	bf16 / 200 steps
DINOv2 input	448×448
Implicit input: simulation / robot	336×336 / 448×448
Policy-overlay threshold	0.7
LIBERO episode cap	400 steps
Replanning interval / DDIM steps	8 / 8

Training uses AdamW, a warm-up followed by cosine learning-rate decay, and gradient clipping. The 100 training diffusion steps and eight DDIM inference steps refer to different stages of the diffusion model and should not be read as competing execution settings. Both simulation and robot policies replan after executing eight steps of an action chunk.

E.3 Simulation and Robot Execution

LIBERO-Object uses 100 expert demonstrations and 50 evaluation episodes per task, with a 400-step cap. Each task requires picking up its named object and placing it in the basket. The aggregate is the mean success rate across the ten tasks; App. F provides the available task-level counts and their corresponding means.

For real-world evaluation, an AgileX Piper six-DoF arm uses head- and wrist-mounted RGB cameras. We collect 100 teleoperated demonstrations per task and train one multi-task policy for each baseline over all four tasks. The RGB front ends use DINOv2-ViT-L/14 with registers at 448×448 . Language features are provided by frozen MiniLM-L6-v2 with dimension 384. The implicit front end uses the frozen grounding model at 448×448 , with the same top-1 region selection and thresholded mask pooling as in simulation.

A ZMQ multi-task action server communicates with the Piper client and replans every eight steps using eight DDIM sampling steps. Each task is evaluated with ten trials, so the reported real-robot aggregate is a success count out of 40. The three-seed averages used for grounding accuracy are not applied to these robot trial counts. The qualitative sequences in App. G illustrate execution; the full outcomes are represented by the trial totals.

F Per-task Policy Results

Object-wise variation. Tab. 12 gives the available per-task breakdown for DP-RGB, UAD-DP, and A2A-Explicit. Its column means match the aggregate results in Tab. 5. For A2A-Implicit, only aggregate success rates are reported.

Table 12. Per-task LIBERO-Object success rates (%) for the RGB baseline and the two explicit-conditioning policies, with 50 episodes per task. Every task picks up the named object and places it in the basket. A2A-Implicit is reported at the aggregate level in Tab. 5.

Target object	DP-RGB	UAD-DP	A2A-Explicit
Alphabet soup	88.0	96.0	96.0
Cream cheese	90.0	50.0	98.0
Salad dressing	92.0	100.0	100.0
Barbecue sauce	72.0	48.0	86.0
Ketchup	88.0	42.0	96.0
Tomato sauce	84.0	86.0	88.0
Butter	88.0	92.0	90.0
Milk	92.0	52.0	92.0
Chocolate pudding	88.0	80.0	100.0
Orange juice	96.0	98.0	98.0
Mean	87.8	74.4	94.4

Explicit conditioning improves cream cheese from 90% to 98%, barbecue sauce from 72% to 86%, and chocolate pudding from 88% to 100%. Milk remains at 92%, showing that the gain is not uniform across objects. UAD-DP is competitive on some tasks, such as salad dressing, but substantially weaker on others; its aggregate should not be read as a failure on every object. These differences motivate reporting task-level scores alongside the mean.

Real-robot task breakdown. Tab. 13 reports ten trials for each of the four tasks. Summing each column gives the corresponding total in Tab. 5. The updated A2A-Implicit aggregate is 25/40; its per-task counts are not available, so no task-level allocation is inferred.

Table 13. Real-robot success counts for the RGB baseline and explicit-conditioning policies. Each task has ten trials. The aggregate A2A-Implicit result is 25/40 (Tab. 5).

Task	DP-RGB	UAD-DP	A2A-Explicit
Open the microwave oven	8/10	7/10	9/10
Stack the blue cube on the wooden cube	6/10	4/10	8/10
Place the cup on the mat	6/10	4/10	7/10
Place the phone on the phone stand	2/10	1/10	2/10
Total	22/40	16/40	26/40

G Real-world Demonstration Sequences

The four sequences show A2A-Implicit, A2A-Explicit, and UAD-DP from top to bottom. Each figure includes its original Start-to-End arrow indicating left-to-right progression. These qualitative examples supplement the trial counts and do not define a separate evaluation set.

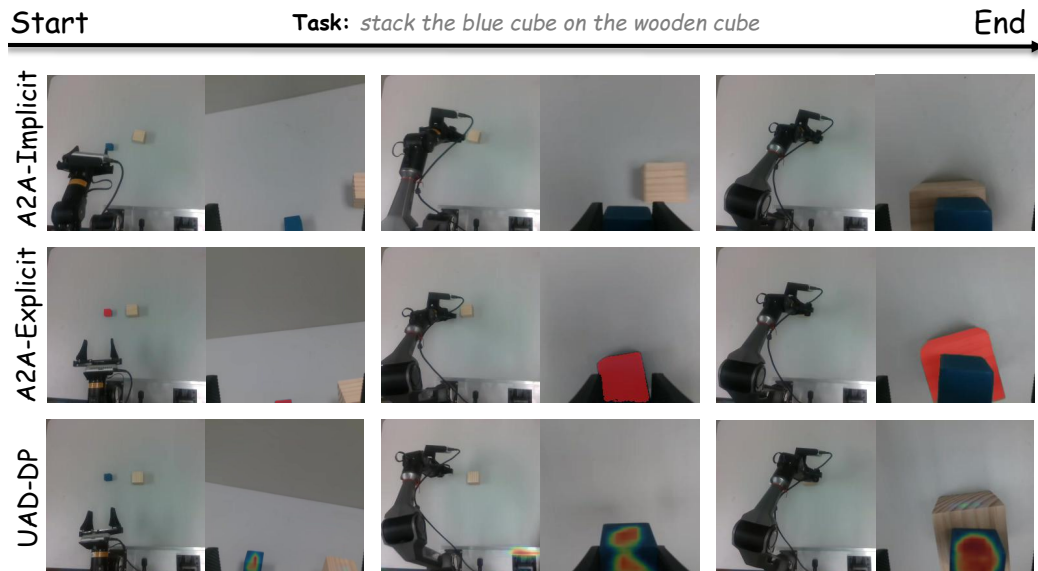


Figure 8. Stack the blue cube on the wooden cube. Rows show A2A-Implicit, A2A-Explicit, and UAD-DP from top to bottom; the embedded Start-to-End arrow indicates temporal progression.

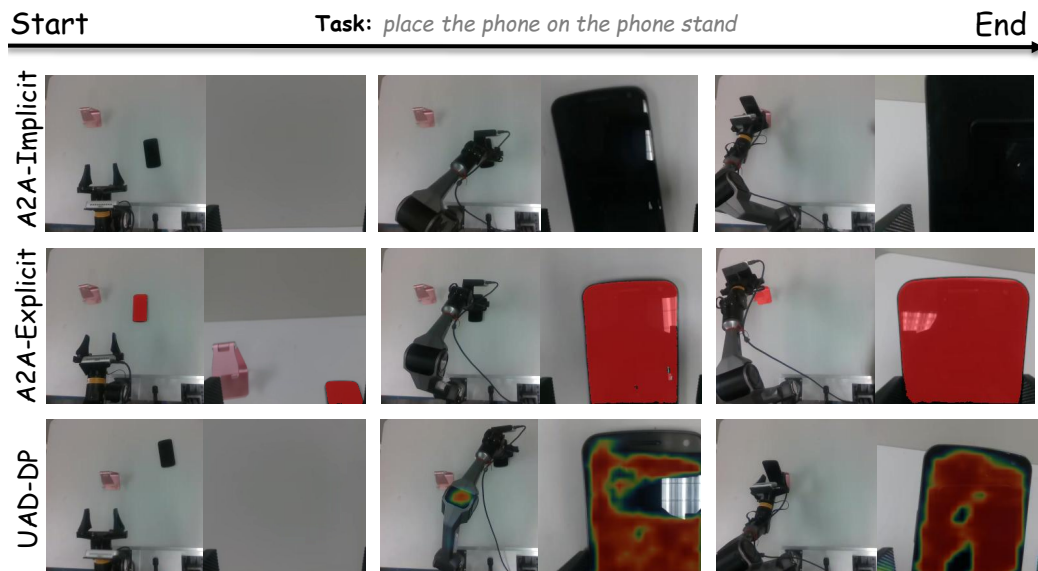


Figure 9. Place the phone on the phone stand. Rows show A2A-Implicit, A2A-Explicit, and UAD-DP from top to bottom; the embedded Start-to-End arrow indicates temporal progression.

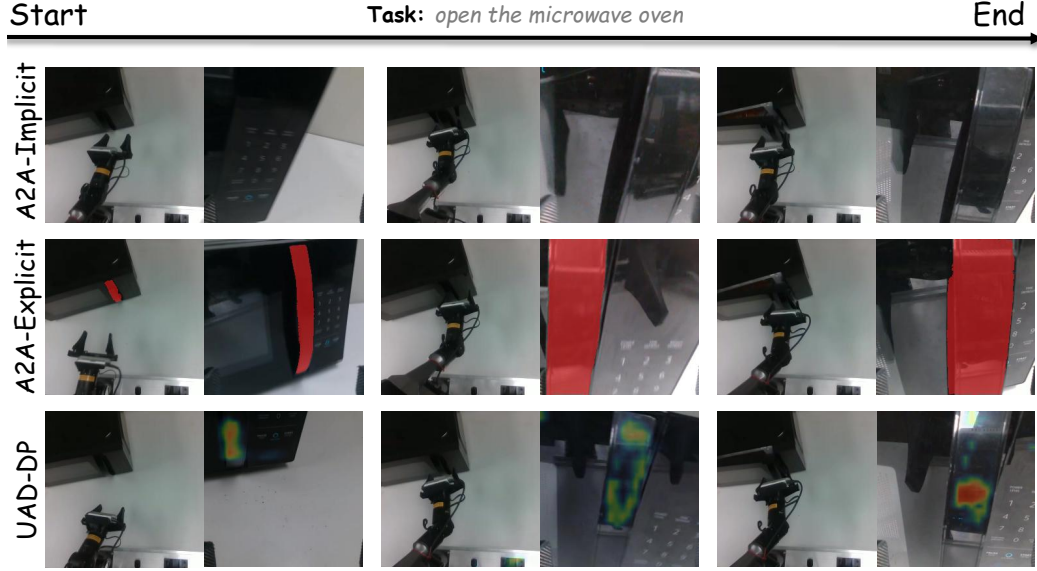


Figure 10. Open the microwave oven. Rows show A2A-Implicit, A2A-Explicit, and UAD-DP from top to bottom; the embedded Start-to-End arrow indicates temporal progression.

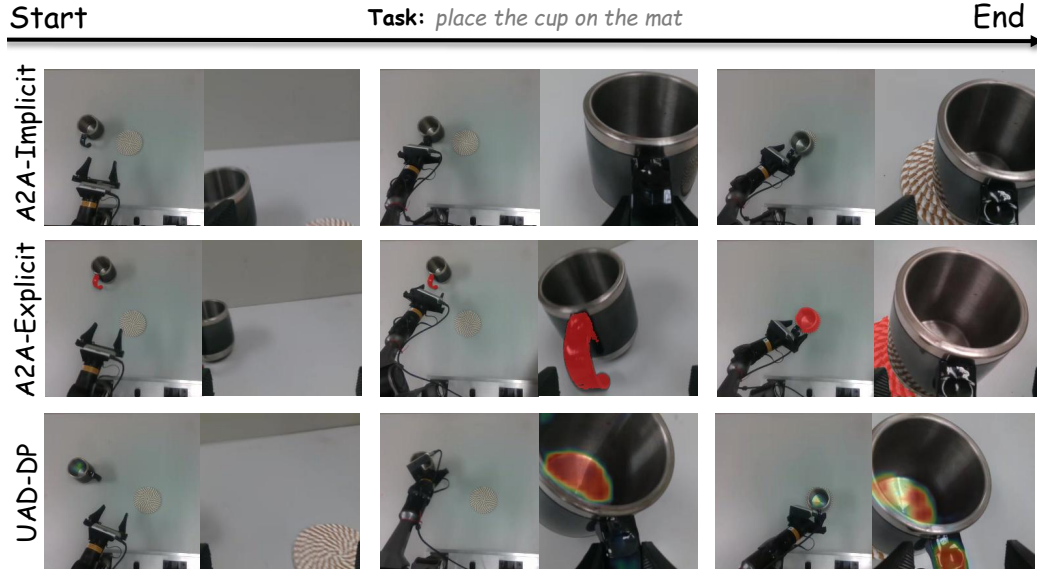


Figure 11. Place the cup on the mat. Rows show A2A-Implicit, A2A-Explicit, and UAD-DP from top to bottom; the embedded Start-to-End arrow indicates temporal progression.